

I F T 2 2 2

# Computer Architecture and Organisation

*Study Manual*

---

Victor Ayodeji

FETCH      1010 0011 1100 1010

DECODE    **ADD** R3, [202]

EXEC      **|** R3 <- R3 + M[202]

float      0 10000100 1000000...      = 64.2

CPU

CACHE

MAIN MEMORY

I/O

# Contents

*Every line is a link. The page numbers are read back from the rendered book, never typed by hand.*

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| How to use this manual .....                                          | 1         |
| <b>FOUNDATIONS · THE MATHS YOU NEED FIRST .....</b>                   | <b>4</b>  |
| F.1 · Number Systems and Positional Notation .....                    | 5         |
| F.2 · Converting Between Bases .....                                  | 7         |
| <b>MODULE ONE · FUNDAMENTALS OF COMPUTER ARCHITECTURE .....</b>       | <b>10</b> |
| Unit 1 · Architecture versus Organization .....                       | 11        |
| Unit 2 · Data Representation .....                                    | 16        |
| Unit 3 · Floating Point Representation .....                          | 24        |
| Unit 4 · Instruction Set Architecture .....                           | 28        |
| <b>MODULE TWO · PROCESSOR DESIGN AND PERFORMANCE .....</b>            | <b>35</b> |
| Unit 1 · Processor Components .....                                   | 36        |
| Unit 2 · CPU Performance Metrics .....                                | 38        |
| Unit 3 · Pipelining .....                                             | 41        |
| <b>MODULE THREE · COMPUTER MEMORY ORGANIZATION .....</b>              | <b>48</b> |
| Unit 1 · Computer Memory Overview .....                               | 49        |
| Unit 2 · Semiconductor Memory .....                                   | 51        |
| Unit 3 · Cache Memory Design .....                                    | 52        |
| <b>MODULE FOUR · BASIC AND ADVANCED PROCESSOR ARCHITECTURES .....</b> | <b>59</b> |
| Unit 1 · RISC versus CISC .....                                       | 60        |
| Unit 2 · Parallel Architectures and Interconnection Networks .....    | 61        |
| <b>SUPPLEMENT · DIGITAL LOGIC .....</b>                               | <b>63</b> |
| S.1 · Boolean Algebra and K-maps .....                                | 64        |
| <b>MOCK EXAMINATION ONE .....</b>                                     | <b>66</b> |
| Question Paper .....                                                  | 67        |
| Answers .....                                                         | 68        |
| <b>MOCK EXAMINATION TWO .....</b>                                     | <b>72</b> |
| Question Paper .....                                                  | 73        |
| Answers .....                                                         | 74        |

## BEFORE YOU START

# How to use this manual

*This book takes you from knowing nothing about how a computer is built to solving every past question on this course with a perfect score. Read it with a pen, not just your eyes.*

Computer Architecture and Organisation looks frightening because it mixes three things at once: a little binary arithmetic, a handful of diagrams, and a set of formulas for speed and memory. In truth the whole course is about a dozen ideas, each used over and over. This manual finds those ideas, teaches each one from zero, and then makes you use it on the real exam questions straight away. Nothing is assumed. Every number is worked out in full. Every symbol is explained before it appears.

## The one habit that makes this work

Reading feels like studying and teaches almost nothing. What moves a fact into your memory is pulling it back out. So all through the book you will meet a **RECALL AND CHECK** box: a question with the answer hidden under a dashed line. Cover the answer, say yours out loud or on paper, then look. When a worked example ends, you will see **Now redo it**: close the book and reproduce the solution on blank paper. If you only ever do that one thing, you will pass.

## The boxes, and what each is for

Every idea lives in a labelled box. You can navigate the whole book by the colour and shape of these boxes alone.

### TEACH

One idea, in plain language. This is where a concept is explained for the first time. Steel-blue, the colour of the datapaths and buses this course is about.

### WORKED EXAMPLE

SLIDE / CLASSWORK / PQ

A full solution, one numbered step at a time, with the reason on every line. The chip on the right tells you where the question came from, so a chip that reads **PQ 25/26** is a real exam question you are being shown how to solve. Amber, the colour of energy and work being done.

### ● MUST MEMORISE

MEMORY HOOK

The few things you truly have to know by heart. This violet is **reserved**: it appears nowhere else in the book, so if you see it, commit it to memory. Each one comes with a hook to make it stick.

### ⚠ TRAP

The exact mistake students make on this topic every year. Read these twice. Losing marks here is the most avoidable way to lose them.

### TRACE

A step-by-step table that follows a value, a register, or a pipeline cycle by cycle, so you can see the machine change state one move at a time.

### RECALL AND CHECK

**What should you do the moment you reach one of these?**

-----  
ANSWER

Cover this line, answer from memory, then check. Retrieval is the whole point.

## FILL IN THE GAP

Short cloze practice: finish the line, then check against the answers underneath.

## EXAM QUESTION

PQ 25/26 Q1A

A past question reproduced as it appeared on the paper. Every one in this book is followed by a full worked answer in a green **Answer** box, so no question on this course goes unanswered.

## How the book is arranged

A short **Foundations** part comes first: the number sense the rest of the course leans on. Then four modules follow the syllabus exactly: Fundamentals, Processor Design and Performance, Memory Organisation, and Advanced Architectures. As each topic is taught, the real past questions on it are solved right there. Two full **mock examinations** at the end are built in the shape of the real paper, answer question one and any other two, seventy marks in two hours, and both are worked in full.

## TEACH

## WHAT THE EXAM LOOKS LIKE

The paper is always the same shape. **Question One is compulsory and carries 30 marks.** You then choose **any two of the remaining questions**, 20 marks each, for a total of 70 in 2 hours. Question One is where the numeric work concentrates: number representation, floating point, cache, pipelining and performance. Master Question One and you have already secured nearly half the paper before you choose anything.

## FOUNDATIONS

# The Maths You Need First

*Everything a computer does, it does with two symbols, 0 and 1. Before we can talk about processors, memory or floating point, we need to be completely at home moving between binary, decimal and hexadecimal. This part builds that fluency from the ground up.*

1010 0011                      =    A3                      =    163

---

binary

hexadecimal

decimal

# Number Systems and Positional Notation

A number system is just a set of symbols and one rule for reading a row of them. Learn the rule once and every base behaves the same way.

## What "base" means

You already use a number system fluently: the decimal system, base 10. It has ten symbols, 0 to 9. When you run out of symbols you do not invent a new one, you start a new column to the left. The genius of the system, invented long before computers, is that **the position of a digit tells you its value**. In 735 the 7 does not mean seven, it means seven hundred, because of where it sits.

| TEACH                                                                                                                                                                                                                                                                                                                        | POSITIONAL NOTATION                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Reading right to left, each column is worth the base raised to a higher power: the ones column is <math>\text{base}^0</math>, the next is <math>\text{base}^1</math>, the next <math>\text{base}^2</math>, and so on. The value of the whole number is each digit multiplied by its column, all added up. In base 10:</p> |                                                                                                                                            |
| $10^2 = 100$<br><div style="border: 1px solid black; padding: 10px; text-align: center; width: 100px; margin: 10px auto;">7</div> <p>700</p>                                                                                                                                                                                 | $10^1 = 10$<br><div style="border: 1px solid black; padding: 10px; text-align: center; width: 100px; margin: 10px auto;">3</div> <p>30</p> |
| $10^0 = 1$<br><div style="border: 1px solid black; padding: 10px; text-align: center; width: 100px; margin: 10px auto;">5</div> <p>5</p>                                                                                                                                                                                     |                                                                                                                                            |
| <p><math>700 + 30 + 5 = 735</math></p>                                                                                                                                                                                                                                                                                       |                                                                                                                                            |
| <p><i>The same rule reads every base. Only the column values change.</i></p>                                                                                                                                                                                                                                                 |                                                                                                                                            |

## The four bases this course uses

Humans count in base 10. Computers are built from switches that are either on or off, so at heart they count in **base 2, binary**, with just two symbols, 0 and 1. Binary numbers get long quickly, so we also use **base 16, hexadecimal** and occasionally **base 8, octal** as a shorthand: they pack neatly into binary, as F.2 will show. A single binary digit is a **bit**; eight bits make a **byte**.

| SYSTEM      | BASE | SYMBOLS        | SUFFIX WE USE      | EXAMPLE         |
|-------------|------|----------------|--------------------|-----------------|
| Binary      | 2    | 0 1            | B, or subscript 2  | 1011B           |
| Octal       | 8    | 0 to 7         | subscript 8        | 13 <sub>8</sub> |
| Decimal     | 10   | 0 to 9         | D, or none         | 735             |
| Hexadecimal | 16   | 0 to 9, A to F | H, or subscript 16 | A34EH           |

## ● MUST MEMORISE

## HEX LETTERS

Hexadecimal needs six extra symbols once it passes 9. Know these cold, both directions:

| HEX     | A  | B  | C  | D  | E  | F  |
|---------|----|----|----|----|----|----|
| DECIMAL | 10 | 11 | 12 | 13 | 14 | 15 |

**Memory hook:** After nine comes ten, and **A** is the first letter, so A is 10. F is the last of the six and the largest single hex digit, 15, one less than the base.

## ● MUST MEMORISE

## POWERS OF TWO

These come up in every cache, memory and address question on the paper. You must know them without counting on your fingers.

| N     | 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7   | 8   | 9   | 10   |
|-------|---|---|---|---|----|----|----|-----|-----|-----|------|
| $2^n$ | 1 | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 |

**Memory hook:**  $2^{10} = 1024$  is "1 kilo" (K). After that they climb in steps of ten:  $2^{20}$  is 1 M (mega),  $2^{30}$  is 1 G (giga). So  $2^{16} = 2^{10} \times 2^6 = 1024 \times 64 = 65\,536$ .

## WORKED EXAMPLE

## SLIDE

Read the binary number 1011B into decimal.

- 1 Write each bit over its column value, right to left: columns are  $2^3$ ,  $2^2$ ,  $2^1$ ,  $2^0$ , that is 8, 4, 2, 1.

*Bits: 1, 0, 1, 1.*

- 2 Keep only the columns with a 1 in them: 8, then 2, then 1.

*The 4 column has a 0, so it contributes nothing.*

- 3 Add them:  $8 + 2 + 1 = 11$ .

1011B =  $8 + 2 + 1 = 11$  (decimal)

**Now redo it:** cover the working and convert 10110B. You should get 22.

## RECALL AND CHECK

**In any base, what is the value of the rightmost column, and why is it the same for every base?**

ANSWER

It is always 1, because the rightmost column is  $\text{base}^0$ , and any base raised to the power 0 is 1. That is why the last digit is called the units digit in every system.

## Converting Between Bases

Four conversions cover everything the exam asks: any base to decimal, decimal to any base, and the quick grouping trick between binary and hex or octal.

### Any base to decimal: multiply and add

This is just positional notation from F.1 applied in reverse. Multiply each digit by its column value and add. It works for any base, you only change what the columns are worth.

#### WORKED EXAMPLE

SLIDE

Convert the hexadecimal number A34EH to decimal.

- 1 Replace each hex letter by its value: A = 10, 3 = 3, 4 = 4, E = 14.
- 2 Write the column values, right to left, as powers of 16:  $16^3 = 4096$ ,  $16^2 = 256$ ,  $16^1 = 16$ ,  $16^0 = 1$ .
- 3 Multiply digit by column:  
 $10 \times 4096 = 40\,960$ ;  $3 \times 256 = 768$ ;  $4 \times 16 = 64$ ;  $14 \times 1 = 14$ .
- 4 Add:  $40\,960 + 768 + 64 + 14 = 41\,806$ .

$$A34EH = 40\,960 + 768 + 64 + 14 = 41\,806 \text{ (decimal)}$$

**Now redo it:** reproduce the four column values from memory before you multiply. That is where marks are lost, not in the adding.

### Decimal to any base: divide and collect remainders

Go the other way by repeatedly dividing by the base and writing down the remainder each time. The remainders, **read from bottom to top**, are the answer.

#### WORKED EXAMPLE

SLIDE

Convert decimal 13 to binary.

| DIVIDE      | QUOTIENT | REMAINDER |
|-------------|----------|-----------|
| $13 \div 2$ | 6        | 1         |
| $6 \div 2$  | 3        | 0         |
| $3 \div 2$  | 1        | 1         |
| $1 \div 2$  | 0        | 1         |

Read the remainders bottom to top: 1101.

$$13 = 1101B \quad (\text{check: } 8 + 4 + 1 = 13)$$

**Now redo it:** convert 22 to binary. You should get 10110, and the check is  $16 + 4 + 2 = 22$ .



### ⚠ TRAP

The remainders come out **least significant first**. If you read them top to bottom you get the number backwards. Always read the column of remainders **from the bottom up**, and sanity-check by converting back.

## Binary to hex, and back: group in fours

Hexadecimal exists to make binary readable, and the reason is exact: **one hex digit is worth exactly four bits** (because  $16 = 2^4$ ). So to turn binary into hex you simply slice the bits into groups of four from the right and convert each group on its own. Octal is the same trick in groups of three, because  $8 = 2^3$ .

### WORKED EXAMPLE

### METHOD

Convert 11010110B to hexadecimal.

- 1 Slice into groups of four bits, starting from the right:  
*1101 0110.*
- 2 Convert each group using the powers 8, 4, 2, 1:  
 *$1101 = 8+4+1 = 13 = D$ ;  $0110 = 4+2 = 6$ .*
- 3 Write the hex digits in the same order: D6.

11010110B = 1101 0110 = D6H (check: both equal 214)

**Now redo it:** go backwards. Convert 2FH to binary by expanding each digit to four bits: 2 = 0010, F = 1111, giving 0010 1111.

### ⚠ TRAP

Group from the **right**, not the left, and pad the leftmost group with leading zeros if it is short. Grouping 110101101011101 from the left would misalign every digit. From the right it is 110 1011 0101 1101, and the short leading group 110 pads to 0110 = 6, giving 6B5DH.

### FILL IN THE GAP

1. The rightmost column in base  $b$  is always worth \_\_\_\_\_.
2. One hexadecimal digit is worth exactly \_\_\_\_\_ bits.
3. To convert decimal to binary you repeatedly divide by 2 and read the \_\_\_\_\_ from bottom to top.
4.  $2^{10} =$  \_\_\_\_\_.

### ANSWERS

1. one ( $b^0 = 1$ ). 2. four. 3. remainders. 4. 1024.

## RECALL AND CHECK

### Why can binary group into hex in fours but not into decimal in any fixed size?

---

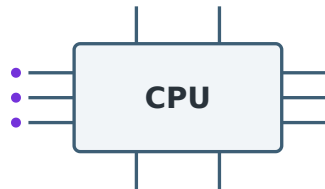
#### ANSWER

Because 16 is a whole power of 2 ( $16 = 2^4$ ), so four bits map exactly onto one hex digit with none left over. 10 is not a power of 2, so no fixed number of bits ever lines up cleanly with one decimal digit. This is the whole reason hex and octal, not decimal, are used as binary shorthand.

## MODULE ONE

# Fundamentals of Computer Architecture

*What a computer is, from the outside in: the difference between what the programmer sees and how the machine is built, the two great design templates every computer follows, and the fetch, decode, execute rhythm that all the later modules refine.*



# Architecture versus Organization

*Two words that sound like synonyms, mean different things, and are worth an easy few marks on every paper if you can tell them apart cleanly.*

*By the end of this unit you can define architecture and organization and give examples of each, draw and compare the Von Neumann and Harvard designs, explain the Von Neumann bottleneck and how to reduce it, and answer every past question of this type in full.*

## TEACH

## WHAT A COMPUTER IS, AND ITS PARTS

A **computer** is a programmable device that stores, retrieves and processes data, following a stored set of instructions to turn input into output. Every computer, from a phone to a server, is built from the same handful of parts:

- ▶ **CPU** (processor): the control unit and ALU that carry out instructions.
- ▶ **Memory**: holds the instructions and data the CPU is working on (Module Three).
- ▶ **Storage**: keeps data when the power is off (disk, SSD).
- ▶ **Input/output devices**: move data in and out (keyboard, screen, network).
- ▶ **The motherboard** (system board) and its **buses**: the circuit board that holds the parts and the wires that join them.

The rest of this module looks at how those parts are organised and how the CPU steps through a program.

## The one distinction everything hangs on

Think of a car. The **architecture** is what the driver sees and uses: the steering wheel, the pedals, the gear positions, the promise that pressing the accelerator makes the car go faster. The **organization** is what is under the bonnet: the size of the engine, how the fuel is delivered, how the pedal is physically connected to the engine. Two cars can drive identically for the driver (same architecture) while being built completely differently inside (different organization).

## TEACH

## THE CORE IDEA

**Computer architecture** is the set of attributes visible to the programmer, the things that affect how a program is written: the instruction set, the number and size of registers, the data types, the addressing modes, the way memory is addressed. It answers "**what does the machine do?**"

**Computer organization** is how those attributes are actually realised in hardware, the parts hidden from the programmer: the control signals, the buses, the memory technology used, whether an add is done by one circuit or several. It answers "**how does the machine do it?**"

**Architecture = what the programmer sees. Organization = how it is built.**

**Memory hook:** Architecture is **Abstract** (the visible promise); Organization is the **Operation** (the hidden wiring that keeps the promise). Same instruction set, different organization, is exactly how a cheap and an expensive processor can run the identical program.

| COMPUTER ARCHITECTURE                                    | COMPUTER ORGANIZATION                                                        |
|----------------------------------------------------------|------------------------------------------------------------------------------|
| Attributes visible to the programmer                     | Attributes hidden from the programmer                                        |
| Answers "what is done"                                   | Answers "how it is done"                                                     |
| Instruction set, data types, registers, addressing modes | Control signals, buses, memory technology, circuits                          |
| Decided first, changes rarely                            | Realises the architecture, varies between models                             |
| Example: the machine has a "multiply" instruction        | Example: multiply is done by repeated addition, or by a dedicated multiplier |

## WORKED EXAMPLE

PQ 25/26 Q1A(I)

"Computer Systems are designed using two closely related but distinct concepts which are Computer Architecture and Computer Organization. Differentiate between these concepts." (2 marks)

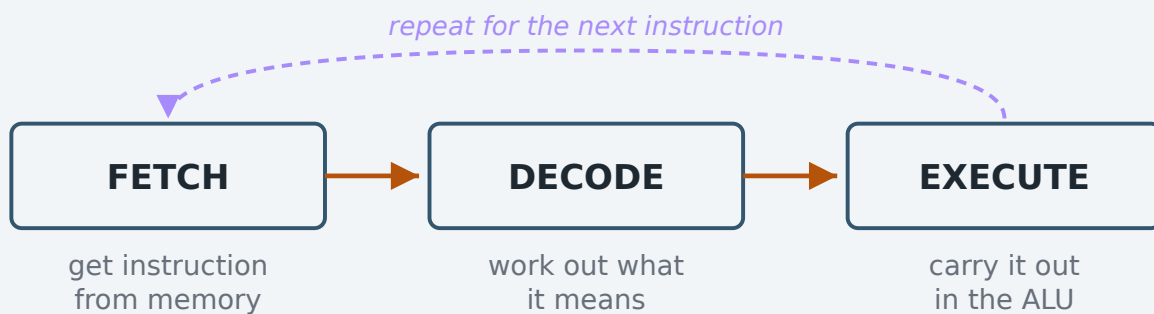
- 1 Name the viewpoint of each: architecture is the programmer's view, organization is the hardware's view.
- 2 Give the defining question each answers, and one example, to earn both marks.

Architecture is the set of attributes visible to the programmer (instruction set, registers, addressing modes): it defines **WHAT** the machine does. Organization is how those attributes are implemented in hardware (control signals, buses, memory technology): it defines **HOW** the machine does it. Example: architecture says a **MULTIPLY** instruction exists; organization decides whether it runs on a dedicated multiplier or by repeated addition.

**Now redo it:** this exact question, worth 2 marks, is on the 24/25 and 25/26 papers. Write the two sentences and one example from memory.

## How a computer runs a program: the instruction cycle

Before we compare designs, fix the basic rhythm in your mind, because every later idea (pipelining, cache, performance) is a refinement of it. The processor repeats three steps, over and over, billions of times a second.



*Fetch, decode, execute, then repeat for the next instruction.*

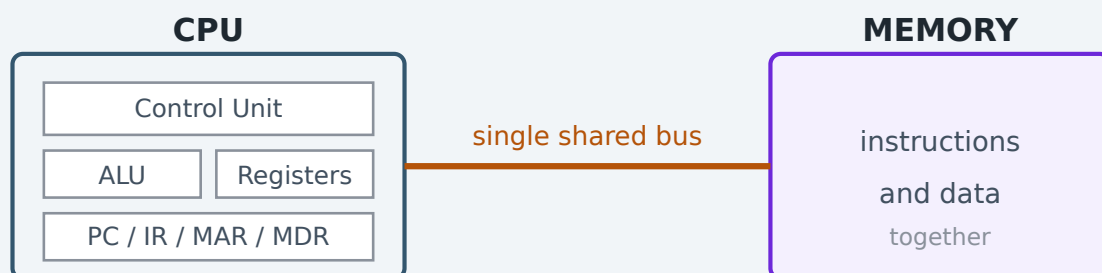
**Fetch:** the control unit reads the next instruction from memory into the processor.

**Decode:** it works out what the instruction is asking for. **Execute:** the arithmetic and logic unit (ALU) carries it out. Then the cycle starts again on the next instruction. Everything else in this course is about doing these steps faster or feeding them with data sooner.

## Two ways to wire memory to the processor

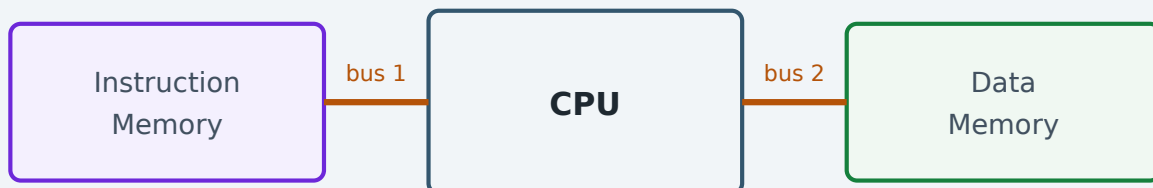
A computer must store both the **program** (the instructions) and the **data** those instructions work on. The two great design templates differ in one decision: do instructions and data share one memory and one path, or get separate ones?

In the **Von Neumann** design there is **one memory** holding both instructions and data, reached over **one shared bus**. This is elegant and cheap: the same memory stores everything, and a program can even treat its own instructions as data. Almost all general-purpose computers use it.



*One memory, one bus: instructions and data share the same road.*

In the **Harvard** design, instructions and data live in **separate memories** reached over **separate buses**. The processor can fetch an instruction and read data **at the same time**, which is faster, but it costs more and is less flexible. It is common in signal processors and microcontrollers, and inside the CPU's own caches.



*Two memories, two buses: an instruction and data can move at once.*

| FEATURE                  | VON NEUMANN               | HARVARD                                         |
|--------------------------|---------------------------|-------------------------------------------------|
| Memory for code and data | One shared memory         | Two separate memories                           |
| Buses                    | One shared bus            | Separate instruction and data buses             |
| Fetch and data access    | Cannot happen at once     | Can happen at the same time                     |
| Cost and complexity      | Lower, simpler            | Higher, more hardware                           |
| Typical use              | General-purpose computers | Signal processors, microcontrollers, CPU caches |

## WORKED EXAMPLE

PQ 25/26 Q1A(II)

"Compare Von Neumann Architecture and Harvard Architecture. Include a diagram for each." (3 marks)

Full-mark answer

Von Neumann uses ONE memory and ONE bus for both instructions and data, so it is simpler and cheaper but the single bus can carry only one of them at a time. Harvard uses SEPARATE memories and buses for instructions and data, so it can fetch an instruction and access data simultaneously (faster) at higher cost. Draw the two diagrams above: one CPU-to-single-memory, one CPU with two memories.

Three marks means three scoring points: one shared versus two separate memories, the simultaneous-access difference, and the two diagrams. Give all three.

**Now redo it:** draw both diagrams from memory in under two minutes. The diagrams alone are worth marks and are the fastest to lose under time pressure.

## The Von Neumann bottleneck

The Von Neumann design has one famous weakness, and the exam asks about it almost every year, so it is worth full marks to you.

### TEACH

### THE BOTTLENECK

Because instructions and data share **one bus**, the processor cannot fetch an instruction and read or write data at the same instant. It must take turns. The single shared path between the CPU and memory therefore limits how fast the whole machine can go, however fast the CPU itself is. That limit is the **Von Neumann bottleneck**: the CPU is often left waiting on the one road to memory.

### TEACH

### HOW TO REDUCE IT

Three standard ways to ease the bottleneck, each worth a mark:

- ▶ **Cache memory:** keep a small, fast copy of the most-used instructions and data inside or beside the CPU, so most accesses never travel the slow bus (Module Three).
- ▶ **Separate caches for instructions and data** (a Harvard-style split inside the chip), so the two can be fetched at once even on a Von Neumann machine.
- ▶ **Wider or faster buses, and registers:** move more bits per trip, and keep frequently used values in fast CPU registers so the bus is used less often.

Modern machines add further tricks in the same spirit: faster **DDR** memory, **prefetching** data into cache before it is asked for, and even **processing-in-memory (PIM)**, doing simple work in the memory chip itself to save a bus trip.

### WORKED EXAMPLE

PQ 24/25 Q3A · PQ 23/24 Q4A

"(i) State the Von Neumann concept. (ii) State two significant aspects of it. (iii) What is the Von Neumann bottleneck?" (1 + 2 + 2 marks)

(i) The Von Neumann concept: a computer stores its program (instructions) and its data together in the SAME memory, and executes instructions one after another by fetching them from that memory. (ii) Two aspects: (a) a single shared memory holds both instructions and data; (b) instructions are executed sequentially, one at a time, using a stored program. (iii) The bottleneck: because one bus is shared for instructions and data, the CPU cannot fetch an instruction and access data simultaneously, so the single CPU-to-memory path limits overall speed.

This same material answers the recurring bottleneck-and-mitigations question on the 20/21 paper: add the three fixes above for the extra marks.

### ⚠ TRAP

Do not say Harvard is "better" and stop there. The exam wants the **trade-off**: Harvard is faster because of simultaneous access, but Von Neumann is simpler, cheaper and more flexible, which is why general-purpose computers use it. A bare "Harvard is faster" leaves marks on the table.



## RECALL AND CHECK

**In one sentence each: what is the Von Neumann bottleneck, and what is the single most common cure for it?**

ANSWER

The bottleneck is that the single shared bus lets only instructions or data move at a time, so the CPU waits on memory. The most common cure is cache memory: a small fast store that keeps most accesses off the slow bus.

## FILL IN THE GAP

1. Architecture describes what the \_\_\_\_\_ sees; organization describes how it is \_\_\_\_\_.
2. Von Neumann uses \_\_\_\_\_ memory for code and data; Harvard uses \_\_\_\_\_.
3. The Von Neumann bottleneck exists because the \_\_\_\_\_ is shared between instructions and data.
4. The three steps of the instruction cycle are fetch, \_\_\_\_\_ and \_\_\_\_\_.

ANSWERS

1. programmer; built. 2. one (shared); two separate memories. 3. bus. 4. decode; execute.

## MODULE ONE • UNIT 2

# Data Representation

*How whole numbers, positive and negative, are stored in a fixed number of bits. This is the single most examined topic on the paper, and every mark here is a mark you can guarantee with practice.*

*By the end of this unit you can represent any integer in unsigned, sign-magnitude, one's complement and two's complement form; state the range of each in  $n$  bits; add and subtract in binary; recognise overflow; use end-around carry; and size an image in memory. Every past question of these types is worked in full.*

## Unsigned integers: the starting point

With  $n$  bits and no sign, you can store the whole numbers from 0 up to  $2^n$  minus 1. Each pattern of bits is just its plain binary value. An 8-bit byte therefore holds 0 to 255, because  $2^8$  minus 1 is 255.

### ● MUST MEMORISE

### UNSIGNED RANGE

In  $n$  bits, unsigned integers run **0 to  $2^n - 1$** . That is  $2^n$  different values in total.

**Memory hook:**  $n$  bits give  $2^n$  patterns; one of them is zero, so the largest is one less than  $2^n$ .

Because each position is worth twice the one to its right, moving every bit **one place left** doubles a number ( $\times 2$ ) and moving it **one place right** halves it ( $\div 2$ ), in binary or hex alike. To halve **3FC** in hex, shift its bits right by one:  $3FC = 0011\ 1111\ 1100$ , shifted right is  $0001\ 1111\ 1110 = \mathbf{1FE}$ . Check in decimal:  $3FC = 1020$ , and  $1020 \div 2 = 510 = 1FE$ ; a left shift of **1FE** returns **3FC**. This is why shift instructions are the fastest way a processor multiplies or divides by powers of two.

## The problem: how do you store a minus sign?

A memory cell holds only 0s and 1s, with nowhere to write a "-". So we spend one of the bits, the leftmost, to carry the sign, and 0 means positive, 1 means negative. There are three ways to do it, and the exam expects all three.

- ▶ **Sign-magnitude:** the leftmost bit is the sign, the remaining bits are the plain size (magnitude) of the number. Simple to read, but it has two zeros (+0 and -0) and makes arithmetic awkward.
- ▶ **One's complement:** to make a number negative, **invert every bit** of its positive form. Also has two zeros, but subtraction becomes addition (with a twist called end-around carry, below).
- ▶ **Two's complement:** invert every bit and **add 1**. This is the one real computers use: it has a single zero, and addition and subtraction work with no special cases.

Seeing one small number in all three side by side fixes the idea. Take **-5** in 4 bits. First write +5 as 0101, then apply each rule.

| VALUE | SIGN-MAGNITUDE     | ONE'S COMPLEMENT      | TWO'S COMPLEMENT            |
|-------|--------------------|-----------------------|-----------------------------|
| +5    | 0101               | 0101                  | 0101                        |
| -5    | 1101               | 1010                  | 1011                        |
| How   | flip sign bit to 1 | invert all bits of +5 | invert all bits, then add 1 |

**To negate in two's complement: invert every bit, then add 1.** The same recipe converts back, so applying it twice returns the original.

Ranges in  $n$  bits:

| SCHEME           | RANGE                                | ZEROS |
|------------------|--------------------------------------|-------|
| Unsigned         | 0 to $2^n - 1$                       | one   |
| Sign-magnitude   | $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$ | two   |
| One's complement | $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$ | two   |
| Two's complement | $-2^{n-1}$ to $+(2^{n-1} - 1)$       | one   |

**Memory hook:** two's complement reaches **one extra negative** value, because it has no wasted second zero. In 8 bits that is  $-128$  to  $+127$ .

### WORKED EXAMPLE

PQ 24/25 Q3B · PQ 23/24 Q4B

Complete the table of Unsigned, Sign-magnitude, One's complement and Two's complement for **204** and **-139**, using **12 bits**.

- 1 Write 204 in 12-bit binary.  $204 = 128 + 64 + 8 + 4$ , so  $204 = 0000\ 1100\ 1100$ .  
*Being positive, 204 looks identical in all four schemes.*
- 2 For  $-139$ , first write  $+139$  in 12 bits:  $139 = 128 + 8 + 2 + 1 = 0000\ 1000\ 1011$ .
- 3 Sign-magnitude of  $-139$ : set the sign bit to 1, keep the magnitude:  $1000\ 1000\ 1011$ .
- 4 One's complement of  $-139$ : invert every bit of  $0000\ 1000\ 1011$ :  $1111\ 0111\ 0100$ .
- 5 Two's complement of  $-139$ : add 1 to the one's complement:  $1111\ 0111\ 0100 + 1 = 1111\ 0111\ 0101$ .

| VALUE | UNSIGNED          | SIGN-MAGNITUDE | ONE'S COMPLEMENT | TWO'S COMPLEMENT |
|-------|-------------------|----------------|------------------|------------------|
| 204   | 0000 1100 1100    | 0000 1100 1100 | 0000 1100 1100   | 0000 1100 1100   |
| -139  | not representable | 1000 1000 1011 | 1111 0111 0100   | 1111 0111 0101   |

**Now redo it:** the third row on the real paper gives a bit pattern and asks its value. Read back  $1111\ 0111\ 0101$  as two's complement: the leading 1 says negative, so invert and add 1 ( $0000\ 1000\ 1011 = 139$ ), giving  $-139$ . That is the same number, proving the recipe reverses itself.

A telemetry system uses a **16-bit two's complement** processor. The values are Temperature =  $-182$ , Pressure =  $487$ , Altitude =  $-1023$ . (i) Give the representable range and say whether each value fits. (ii) Represent each in binary and hexadecimal. (iii) Compute  $(-182) + 487$ .

(i) Range

16-bit two's complement:  $-2^{15}$  to  $+(2^{15}-1) = -32768$  to  $+32767$ . All three values ( $-182$ ,  $487$ ,  $-1023$ ) fall inside this range, so all are representable.

(ii) Binary and hex

| VALUE   | 16-BIT TWO'S COMPLEMENT (BINARY) | HEX  |
|---------|----------------------------------|------|
| $-182$  | 1111 1111 0100 1010              | FF4A |
| $487$   | 0000 0001 1110 0111              | 01E7 |
| $-1023$ | 1111 1100 0000 0001              | FC01 |

Method for  $-182$ :  $+182 = 0000\ 0000\ 1011\ 0110$ ; invert to  $1111\ 1111\ 0100\ 1001$ ; add 1 to get  $1111\ 1111\ 0100\ 1010 = \text{FF4A}$ .

(iii)  $(-182) + 487$

Add the 16-bit patterns of  $-182$  (FF4A) and  $487$  (01E7):  $1111\ 1111\ 0100\ 1010 + 0000\ 0001\ 1110\ 0111 = 1\ 0000\ 0001\ 0011\ 0001$ . The carry out of the 16th bit is discarded in two's complement, leaving  $0000\ 0001\ 0011\ 0001 = 0131\text{H} = 305$ . Check:  $-182 + 487 = 305$ . Correct.

**Now redo it:** this whole question is 25/26 Q1c and is worth easy marks. The trick is that in two's complement you just add the patterns and throw away the final carry.

## Adding in binary, and when it overflows

Binary addition is column addition with carries, exactly like decimal but with only 0 and 1. The danger is **overflow**: when the true sum needs more bits than the register has, the top bit is lost and the stored answer is wrong.

## WORKED EXAMPLE

PQ 20/21 Q5B

Add **214 + 65** in an 8-bit register. Give the result in binary and decimal, and comment on overflow.

- 1 Convert:  $214 = 1101\ 0110$ ,  $65 = 0100\ 0001$ .
- 2 Add the two bytes:  $1101\ 0110 + 0100\ 0001 = 1\ 0001\ 0111$ .  
*The result needs 9 bits.*
- 3 An 8-bit register holds only the low 8 bits,  $0001\ 0111 = 23$ , and sets a carry.

True sum  $214 + 65 = 279$ . In 8 bits the register keeps  $0001\ 0111 = 23$  and raises a carry out of the top bit. This is unsigned **overflow**: 279 will not fit in a byte (max 255), so the stored 23 is wrong by exactly  $2^8 = 256$ .

**Now redo it:** overflow is simply "the answer needed more bits than the box has". State the true sum, the wrapped value, and that a carry came out.

## Subtracting by adding: end-around carry

Computers subtract by adding a negative. In **one's complement**, this needs one extra step: if a carry comes out of the top after the addition, you **add it back** into the lowest bit. That step is the **end-around carry**, and it is asked almost every year.

## WORKED EXAMPLE

PQ 23/24, 24/25, TESTS (RECURRING)

Use end-around carry in one's complement to compute **153 – 142** in a 10-bit register.

- 1  $153 - 142$  becomes  $153 + (-142)$ . Write 153 in 10 bits:  $00\ 1001\ 1001$ .
- 2 Write  $-142$  in one's complement:  $+142 = 00\ 1000\ 1110$ , invert to  $11\ 0111\ 0001$ .
- 3 Add:  $00\ 1001\ 1001 + 11\ 0111\ 0001 = 1\ 00\ 0000\ 1010$ .  
*A 1 has carried out of the 10th bit.*
- 4 End-around carry: drop that carry and add it into the lowest bit:  $00\ 0000\ 1010 + 1 = 00\ 0000\ 1011$ .
- 5 Read the result:  $00\ 0000\ 1011 = 11$ . Positive (leading 0), and  $153 - 142 = 11$ . Correct.

$153 - 142 = 00\ 0000\ 1011$  (one's complement, 10 bits) = 11.

**Now redo it:** the signature of one's complement is that a carry out never vanishes, it comes back in at the bottom. If there is no carry out, there is nothing to add and the result is already negative.

## TEACH

## OVERFLOW VERSUS UNDERFLOW

Two words the exam likes to separate. **Overflow** is a result too large (too far positive, or a magnitude too big) to fit the available bits. **Underflow** is a result too small to represent: for integers, a value more negative than the smallest storable number; for floating point (Unit 3), a value so close to zero it rounds away to nothing. Example: in 8-bit two's complement (range  $-128$  to  $+127$ ),  $100 + 50 = 150$  overflows;  $-100 - 50 = -150$  underflows.

## Sizing an image in memory

An image is a grid of dots (pixels). Its size in memory is simply the number of pixels times the bits used per pixel. The number of pixels is the physical size in inches times the resolution in dots per inch (dpi), in each direction.

### ● MUST MEMORISE

### IMAGE SIZE

**bits = (width×dpi) × (height×dpi) × bits-per-pixel.** Then divide by 8 for bytes, by 1024 for KB, by 1024 again for MB. Unless a colour depth is given, this course assumes a black-and-white image at **1 bit per pixel** (as the lecture example does).

### WORKED EXAMPLE

### PQ 23/24, 24/25, 25/26 Q4C (RECURRING)

Find the memory needed for an image 17.12 inches by 9.45 inches at 250 dpi. Give the answer in megabytes.

- 1 Pixels across:  $17.12 \times 250 = 4280$ . Pixels down:  $9.45 \times 250 = 2362.5$ .  
*The height gives a non-whole 2362.5; carry it as is, the arithmetic is what earns the marks.*
- 2 Total pixels:  $4280 \times 2362.5 = 10\,111\,500$ .
- 3 At 1 bit per pixel that is 10 111 500 bits. Divide by 8: 1 263 937.5 bytes.
- 4 Divide by 1024 twice:  $1\,263\,937.5 \div 1\,048\,576 = 1.205$  MB (about 1.21 MB).

10 111 500 pixels × 1 bit = 10 111 500 bits = 1 263 937.5 bytes ≈ **1.21 MB** (dividing by 1024×1024). If the paper divides by one million instead, it is about 1.26 MB: state which you used.

**Now redo it:** if a colour depth were given (say 24-bit colour), you would multiply the pixel count by 24 before dividing. Always check whether the question gives bits per pixel.

## Two ways to store a picture: bitmap and vector

The sizing above assumed a **bitmap**. There are two families of computer graphics, and the point the exam likes is why their sizes differ so much.

### TEACH

### BITMAP VERSUS VECTOR GRAPHICS

- ▶ **Bitmap (raster)** graphics store the picture as a grid of **pixels**, each pixel's colour held explicitly. More pixels per inch means higher **resolution** and a sharper image, but the size grows with the pixel count (as sized above), and enlarging a bitmap makes the individual pixels show. Photographs are bitmaps.
- ▶ **Vector** graphics store the picture as **objects**, such as lines, curves and shapes, each described by attributes: coordinates, length and breadth, line thickness, colour and fill. The image is redrawn from the maths at any size with no loss, and the file holds a few objects rather than millions of pixels, so line art and logos are far smaller as vectors. A photograph, having no simple shapes, does not reduce to vectors.

A black-and-white bitmap is 4 inches by 6 inches at 300 dpi. How much memory does it need?

- 1 Pixels across:  $4 \times 300 = 1200$ . Pixels down:  $6 \times 300 = 1800$ .
- 2 Total pixels:  $1200 \times 1800 = 2\,160\,000$ .
- 3 At 1 bit per pixel (black or white):  $2\,160\,000 \text{ bits} \div 8 = 270\,000 \text{ bytes}$ .
- 4 In KB:  $270\,000 \div 1024 = \mathbf{263.67 \text{ KB}}$ .

$2\,160\,000 \text{ pixels} \times 1 \text{ bit} = 2\,160\,000 \text{ bits} = 270\,000 \text{ bytes} \approx \mathbf{263.67 \text{ KB}}$ . Same method as the 17.12×9.45 image, different numbers.

## Coded decimal: BCD and Excess-3

Pure binary is not always convenient. When decimal digits must stay exact and be read back one digit at a time (tills, meters, calculator displays), each digit is coded on its own.

### TEACH

### BINARY CODED DECIMAL AND WEIGHTED CODES

**Binary Coded Decimal (BCD)** stores each decimal digit in its own 4 bits, so 47 is **0100 0111**, not the pure-binary 101111. It wastes space (four bits can hold 0 to 15 but only 0 to 9 are used) yet keeps decimal exact. Standard BCD is a **weighted** code: the four positions carry the weights **8 4 2 1**, and a digit is whichever bits sum to it. Other weightings exist; the lectures name **5421** and **4221**, where the same "sum the weights of the 1-bits" rule applies.

### WORKED EXAMPLE

### SLIDE: ENCODE 9.2

Encode **9.2** in 8421 (standard BCD), 5421 and 4221.

| DIGIT | 8421       | 5421       | 4221           |
|-------|------------|------------|----------------|
| 9     | 1001 (8+1) | 1100 (5+4) | 1111 (4+2+2+1) |
| 2     | 0010 (2)   | 0010 (2)   | 0010 (2)       |

$9.2 = \mathbf{1001.0010}$  in 8421,  $\mathbf{1100.0010}$  in 5421,  $\mathbf{1111.0010}$  in 4221. Each code splits at the point exactly where decimal does.

### TEACH

### EXCESS-3, A SELF-COMPLEMENTING CODE

**Excess-3 (XS-3)** codes a digit as its 4-bit binary **plus 3**, so 0 is 0011, 5 is 1000, 9 is 1100. It is **non-weighted** but **self-complementing**: the code of  $(9 - d)$  is the bit-by-bit complement of the code of  $d$ , which makes nine's-complement subtraction easy. For instance XS-3 of 6 is 1001 and XS-3 of 3 is 0110, which are exact complements.

| DIGIT | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    |
|-------|------|------|------|------|------|------|------|------|------|------|
| XS-3  | 0011 | 0100 | 0101 | 0110 | 0111 | 1000 | 1001 | 1010 | 1011 | 1100 |

## WORKED EXAMPLE

SLIDE: EXCESS-3 BOTH WAYS

(i) Encode **24** in Excess-3. (ii) Decode **0100 0110** from Excess-3.

**1** Encode 24: digit 2  $\rightarrow 2+3 = 5 = 0101$ ; digit 4  $\rightarrow 4+3 = 7 = 0111$ . So 24 = **0101 0111**.

**2** Decode 0100 0110: 0100 = 4, minus 3 = 1; 0110 = 6, minus 3 = 3. So it reads **13**.

24 in Excess-3 is **0101 0111**; the code 0100 0110 decodes to **13**. Going in, add 3 to each digit; coming out, subtract 3 from each nibble.

## TEACH

CHARACTER CODES: MORSE TO UNICODE

Text is stored by an agreed table mapping each symbol to a number. Historically there was **Morse code** (variable-length dots and dashes, common letters kept short) and **Hollerith code** (punched-card columns, the ancestor of data processing). In computers today:

- ▶ **ASCII** uses 7 bits, so 128 codes (0 to 127). Codes 32 to 126 are printable, code 32 is space and code 127 is DEL; 0 to 31 are control codes. "Extended ASCII" uses the eighth bit for codes 128 to 255 (for example ISO 8859-1, Latin-1).
- ▶ **EBCDIC** uses 8 bits: IBM's mainframe code, descended from punched cards.
- ▶ **Unicode** uses up to 32 bits, one code for every writing system; its first 128 codes are identical to ASCII.

A few ASCII values worth carrying into the exam: 'A' = 65, 'a' = 97, '0' = 48, space = 32.

## ⚠ TRAP

Three classic slips. (1) In two's complement subtraction you **discard** the final carry; in one's complement you **add it back**. Do not mix them up. (2) Sign-magnitude and one's complement have **two** zeros; two's complement has **one**, which is why its range reaches one more negative value. (3) When negating, invert **all** the bits including leading zeros, not just the ones that are set.

## RECALL AND CHECK

**In  $n$  bits, what is the two's complement range, and why is it not symmetric about zero?**

ANSWER

$-2^{n-1}$  to  $+(2^{n-1}-1)$ . It is asymmetric because there is a single zero on the positive side of the wheel, so the negatives get one extra value. In 8 bits:  $-128$  to  $+127$ .



## FILL IN THE GAP

1. To negate a number in two's complement, invert every bit and then add \_\_\_\_\_.
2. Unsigned n-bit integers range from 0 to \_\_\_\_\_.
3. In one's complement subtraction, a carry out of the top is \_\_\_\_\_ to the lowest bit.
4. Overflow means the result needs \_\_\_\_\_ bits than the register has.

## ANSWERS

1. one. 2.  $2^n - 1$ . 3. added back (end-around carry). 4. more.

## MODULE ONE · UNIT 3

# Floating Point Representation

*How a computer stores numbers with a fractional part, using the IEEE-754 standard. Convert cleanly in both directions and you own a whole exam question, because one appears on every paper.*

*By the end of this unit you can lay out the IEEE-754 single-precision format, convert a decimal number to its 32-bit hex form, convert a hex pattern back to decimal, and compare stored floating-point values. Every past question of this type is worked in full.*

## Why floating point, and the idea behind it

Fixed integers cannot store 3.14 or 0.001. Floating point borrows scientific notation: write a number as a sign, a fraction between 1 and 2 (the **mantissa**), and a power of two (the **exponent**), just as  $6.02 \times 10^{23}$  does in base 10. The standard that fixes exactly how those three parts are packed into bits is **IEEE-754**.

## TEACH

## IEEE-754 SINGLE PRECISION

Single precision uses **32 bits** in three fields:



*Sign, then an 8-bit biased exponent, then a 23-bit mantissa.*

- ▶ **Sign (1 bit):** 0 for positive, 1 for negative.
- ▶ **Exponent (8 bits):** the power of two, stored with a **bias of 127** added, so a true exponent of 10 is stored as 137. The bias lets negative exponents be stored without a second sign.
- ▶ **Mantissa (23 bits):** the fraction after the leading 1. Because a normalised number always starts "1.", that 1 is not stored: it is the **hidden bit**, giving 24 bits of precision for free.

Single precision is **1 sign, 8 exponent, 23 mantissa**, bias **127**. To convert a decimal number:

1. Sign bit: 0 if positive, 1 if negative. Work with the size only from here.
2. Write the number in binary (integer part and fraction part).
3. Normalise to  $1.m \times 2^e$  by moving the point.
4. Exponent field =  $e + 127$ , in 8 bits.
5. Mantissa = the bits of  $m$  after the point, filled to 23 bits.

**Memory hook: S, E, M is 1, 8, 23**, and the bias is  $127 = 2^7 - 1$ . Store exponent plus 127; read exponent minus 127.

## WORKED EXAMPLE

## TEST 1 (RECURRING)

Represent **−452.25** in IEEE-754 single precision. This one terminates, so it converts exactly.

- 1 Sign: the number is negative, so the sign bit is **1**.
- 2 Binary:  $452 = 111000100$ , and  $0.25 = 0.01$ , so  $452.25 = 111000100.01$ .
- 3 Normalise: move the point 8 places left,  $111000100.01 = 1.1100010001 \times 2^8$ . So the true exponent is 8.
- 4 Exponent field:  $8 + 127 = 135 = 1000\ 0111$ .
- 5 Mantissa: the bits after the point,  $1100010001$ , padded to 23 bits:  $1100\ 0100\ 0100\ 0000\ 0000\ 000$ .
- 6 Assemble sign, exponent, mantissa and group into hex:  $1\ 1000\ 0111\ 1100\ 0100\ 0100\ 0000\ 0000\ 000 = 1100\ 0011\ 1110\ 0010\ 0010\ 0000\ 0000\ 0000$ .

$-452.25 = 1\ 10000111\ 1100010001000000000000 = \mathbf{C3E22000H}$  (check: converting C3E22000H back gives  $-452.25$  exactly).

**Now redo it:** the five steps never change. Sign, binary, normalise, exponent plus 127, mantissa to 23 bits.

Represent **−1240.56** in IEEE-754 single precision. Here the fraction does not terminate, so we truncate the mantissa at 23 bits.

- 1 Sign: negative, so **1**.
- 2 Binary:  $1240 = 100\ 1101\ 1000$ . The fraction 0.56 keeps going:  $0.56 = 0.10001111010111\dots$  (multiply by 2 repeatedly, taking the whole part each time). So  $1240.56 = 10011011000.10001111010111\dots$
- 3 Normalise:  $1.001101100010001111010111\dots \times 2^{10}$ . True exponent is 10.
- 4 Exponent field:  $10 + 127 = 137 = 1000\ 1001$ .
- 5 Mantissa: the first 23 bits after the point: **0011 0110 0010 0011 1101 011**.

$-1240.56 \approx 1\ 10001001\ 00110110001000111101011 = \mathbf{C49B11EBH}$  (truncated). Rounding the last bit up instead gives **C49B11ECH**, which is what the hardware actually stores. Either is acceptable if you state your method.

**Now redo it:** the only new skill is turning a non-terminating fraction into bits by repeated multiplication by 2, stopping once you have 23 mantissa bits.

## The reverse: hex pattern back to a decimal value

Going backwards, split the 32 bits into the three fields, undo the bias, restore the hidden 1, and evaluate.

### ● MUST MEMORISE

### THE RECIPE, IEEE TO DECIMAL

1. Expand the hex to 32 bits and split as 1, 8, 23.
2. Sign bit gives the sign.
3. True exponent  $e = (\text{exponent field}) - 127$ .
4. Value =  $\pm 1.\text{mantissa}$  (in binary)  $\times 2^e$ .

**WORKED EXAMPLE**
**PQ 24/25 Q4C · PQ 25/26 Q5D**

Find the decimal value of **C4F2E000H** under IEEE-754 single precision.

- 1 Expand to 32 bits: C4F2E000 = 1100 0100 1111 0010 1110 0000 0000 0000.
- 2 Split: sign = 1; exponent = 1000 1001; mantissa = 111 0010 1110 0000... (rest zeros).
- 3 Sign 1 means negative.
- 4 Exponent: 1000 1001 = 137, so  $e = 137 - 127 = 10$ .
- 5 Restore the hidden 1:  $1.11100101110 \times 2^{10}$ . Move the point 10 places right: 11110010111.0.
- 6 Evaluate  $11110010111 = 1024 + 512 + 256 + 128 + 16 + 4 + 2 + 1 = 1943$ .

$$\text{C4F2E000H} = -1.1110010111 \times 2^{10} = -11110010111_2 = \mathbf{-1943}.$$

**Now redo it:** the steps that lose marks are forgetting the hidden 1 and forgetting to subtract 127. Do both, every time.

**WORKED EXAMPLE**
**PQ 25/26 Q4A · PQ 24/25 Q1A · PQ 23/24 Q1C**

Three IEEE-754 numbers are stored: **X = C1400000H**, **Y = 42100000H**, **Z = 41400000H**. Convert each to decimal, then test: (i)  $(Y - Z) > 0$ , (ii)  $X + Z = 0$ , (iii)  $Z = X + Y$ . This 9-mark question recurs on three papers.

Convert each

| HEX      | SIGN / EXP FIELD / E | 1.MANTISSA $\times 2^E$      | DECIMAL |
|----------|----------------------|------------------------------|---------|
| C1400000 | 1 / 130 / 3          | $-1.1 \times 2^3 = -1100$    | -12     |
| 42100000 | 0 / 132 / 5          | $1.0001 \times 2^5 = 100100$ | 36      |
| 41400000 | 0 / 130 / 3          | $1.1 \times 2^3 = 1100$      | 12      |

Apply the tests

$X = -12$ ,  $Y = 36$ ,  $Z = 12$ . (i)  $Y - Z = 36 - 12 = 24 > 0$ : **TRUE**. (ii)  $X + Z = -12 + 12 = 0$ : **TRUE**. (iii)  $X + Y = -12 + 36 = 24$ , which is not 12, so  $Z = X + Y$  is **FALSE**.

**Now redo it:** convert all three first, then the three tests are one line of ordinary arithmetic each. Do not try to compare the hex directly.

## TEACH

## DOUBLE PRECISION, AND WHAT IEEE-754 DEFINES

The same layout scaled up gives **double precision**, used wherever single precision's roughly seven significant digits are not enough, as in banking and scientific work.

| FORMAT | TOTAL   | SIGN | EXPONENT (BIAS) | MANTISSA | APPROX. RANGE  |
|--------|---------|------|-----------------|----------|----------------|
| Single | 32 bits | 1    | 8 (bias 127)    | 23       | $\pm 10^{38}$  |
| Double | 64 bits | 1    | 11 (bias 1023)  | 52       | $\pm 10^{308}$ |

The method is identical; only the field widths and the bias change, double using **bias 1023 =  $2^{10} - 1$** . Beyond the formats, IEEE-754 also fixes the **operations** every conforming machine must provide: add, subtract, multiply, divide, remainder and square root, conversion between decimal and binary, and the handling of exceptions (overflow, underflow, divide-by-zero, and invalid results such as  $0 \div 0$ ).

## ⚠ TRAP

The three marks students throw away here. (1) The exponent is **biased**: subtract 127 when reading, add 127 when writing. (2) The leading 1 is **hidden**, so a mantissa of 111001... means the value 1.111001..., not 0.111001.... (3) When the decimal fraction does not terminate, keep multiplying by 2 until you have 23 mantissa bits, then stop; do not round the whole number off early.

## RECALL AND CHECK

**An IEEE-754 single number has exponent field 1000 0001. What is its true exponent, and why?**

ANSWER

$1000\ 0001 = 129$ , and the true exponent is  $129 - 127 = 2$ . The stored field always carries the bias of 127, so you subtract it to recover the real power of two.

## FILL IN THE GAP

1. IEEE-754 single precision splits 32 bits as 1 sign, \_\_\_\_\_ exponent, \_\_\_\_\_ mantissa.
2. The exponent bias in single precision is \_\_\_\_\_.
3. The leading 1 of a normalised mantissa is not stored; it is called the \_\_\_\_\_ bit.
4. To read the true exponent you \_\_\_\_\_ 127 from the exponent field.

ANSWERS

1. 8; 23. 2. 127. 3. hidden (implicit). 4. subtract.

## MODULE ONE · UNIT 4

# Instruction Set Architecture

*The instruction set is the vocabulary the processor understands, and the boundary between hardware and software. Two exam staples live here: writing an expression in the 0, 1, 2 and*

3 address formats, and computing an effective address.

By the end of this unit you can list the parts of an instruction set architecture, lay out an instruction format, translate an arithmetic expression into the four address organisations and count their bytes, and compute the effective address and operand for every addressing mode.

## What the ISA is, and its parts

The **Instruction Set Architecture (ISA)** is the complete set of instructions a processor can execute, together with everything a programmer needs to use them. It is exactly the "architecture" of Unit 1: the programmer-visible design. Its components:

### TEACH

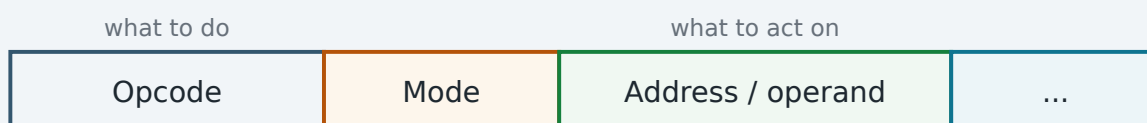
### COMPONENTS OF AN ISA

- ▶ **Instruction set:** the operations available (add, load, jump, and so on).
- ▶ **Registers:** the small fast stores inside the CPU. General-purpose registers hold working values; special ones include the **PC** (program counter, address of the next instruction), **IR** (instruction register, the current instruction), **MAR** (memory address register) and **MDR** (memory data register).
- ▶ **Data types:** the kinds of value handled (integers, floating point, characters, and Booleans, meaning true/false).
- ▶ **Addressing modes:** the ways an instruction can name where its operand is.
- ▶ **Instruction format:** how the bits of an instruction are laid out.

### TEACH

### INSTRUCTION FORMAT

An instruction is a row of bits split into fields: an **opcode** saying what to do, zero or more **operand or address** fields saying what to do it to, and sometimes a **mode** field saying how to read the address.



*Opcode plus operands. The number of address fields names the format.*

An instruction is just a number, so with the format you can read one back. Suppose a 16-bit machine uses **opcode 4 bits, register 4 bits, address 8 bits**, and its opcode table has 1010 = ADD.

- 1 Decode **1010 0011 11001010**. Split 4 / 4 / 8: opcode 1010, register 0011, address 11001010.
- 2 Opcode 1010 = ADD; register 0011 = R3; address 11001010 =  $128+64+8+2 = 202$ .
- 3 So the instruction reads **ADD R3, [202]**: add the contents of location 202 to R3.

A 32-bit three-address machine just widens the fields. With **opcode 6 bits then three 5-bit register fields**, the bits **000001 00001 00010 00011** split as opcode 000001, then R1, R2, R3, giving **ADD R1, R2, R3**.

Reading an instruction is the reverse of building one: know the field widths, chop the bits, translate each field. 1010 0011 11001010 = ADD R3, [202].

## Zero, one, two and three address formats

Processors differ in how many operands one instruction may name. Fewer addresses means shorter instructions but more of them. Learn the four by translating one expression. Take **X = (A + B) × (C + D)**.

## TEACH

## THE SAME EXPRESSION, FOUR WAYS

| 3-ADDRESS<br>OP D,S1,S2 | 2-ADDRESS<br>OP D,S | 1-ADDRESS<br>ACCUMULATOR | 0-ADDRESS<br>STACK |
|-------------------------|---------------------|--------------------------|--------------------|
| ADD T1,A,B              | MOV T1,A            | LOAD A                   | PUSH A             |
| ADD T2,C,D              | ADD T1,B            | ADD B                    | PUSH B             |
| MUL X,T1,T2             | MOV T2,C            | STORE T1                 | ADD                |
|                         | ADD T2,D            | LOAD C                   | PUSH C             |
|                         | MUL T1,T2           | ADD D                    | PUSH D             |
|                         | MOV X,T1            | MUL T1                   | ADD                |
|                         |                     | STORE X                  | MUL                |
|                         |                     |                          | POP X              |
| 3 instructions          | 6 instructions      | 7 instructions           | 8 instructions     |

The pattern: three addresses name both sources and the destination, so each operation is one instruction. Two addresses reuse the destination as a source. One address keeps a running value in the **accumulator** and names only the other operand. Zero addresses take everything from a **stack**, so the expression is written in postfix (reverse Polish) order first.

Fewer address fields, more instructions:

- ▶ **3-address:** op dest, src1, src2. Shortest programs.
- ▶ **2-address:** op dest, src (dest is also a source).
- ▶ **1-address:** LOAD, STORE, and ops against the single **accumulator**.
- ▶ **0-address:** PUSH and POP a **stack**; operators take the top items. Write the expression in **postfix** first.

**Memory hook:** 3, 2, 1, 0 addresses map to register, register, accumulator, stack machines. Postfix of A op B is "A B op".

### WORKED EXAMPLE

PQ 25/26 Q4B · PQ 24/25 Q1D

Write  $X = ((A \times B) + C / D) / (F \times G + J \times K)$  in the 0, 1, 2 and 3 address formats, and count the bytes. Take **1 byte per opcode and 2 bytes per memory address** (state whatever the paper gives).

Instruction counts

| FORMAT    | INSTRUCTIONS | BYTES (OPCODE 1, ADDRESS 2)                                                        |
|-----------|--------------|------------------------------------------------------------------------------------|
| 3-address | 7            | $7 \times (1 + 3 \times 2) = 7 \times 7 = 49$                                      |
| 2-address | 12           | $12 \times (1 + 2 \times 2) = 12 \times 5 = 60$                                    |
| 1-address | 17           | $17 \times (1 + 1 \times 2) = 17 \times 3 = 51$                                    |
| 0-address | 16           | $8 \text{ pushes} \times 3 + 1 \text{ pop} \times 3 + 7 \text{ ops} \times 1 = 34$ |

3-address program (the shortest)

```

MUL  T1, A, B      ; T1 = A*B
DIV   T2, C, D      ; T2 = C/D
ADD   T1, T1, T2    ; T1 = A*B + C/D  (numerator)
MUL   T2, F, G      ; T2 = F*G
MUL   T3, J, K      ; T3 = J*K
ADD   T2, T2, T3    ; T2 = F*G + J*K  (denominator)
DIV   X,  T1, T2    ; X = numerator / denominator

```

0-address program (postfix A B \* C D / + F G \* J K \* + /)

```

PUSH A   PUSH B   MUL           ; A*B on stack
PUSH C   PUSH D   DIV   ADD      ; + C/D -> numerator
PUSH F   PUSH G   MUL           ; F*G
PUSH J   PUSH K   MUL   ADD      ; + J*K -> denominator
DIV      POP X                    ; divide, store X

```

**Now redo it:** the marks are in the counts and one clean listing. Note that fewer addresses always trade shorter instructions for more of them, and the byte totals here are close precisely because of that trade.

## Addressing modes: where the operand really is

An address field does not always hold the operand directly. The **addressing mode** says how to turn that field into the **effective address (EA)**, the actual location of the operand.



| MODE              | EFFECTIVE ADDRESS                   | OPERAND IS                            |
|-------------------|-------------------------------------|---------------------------------------|
| Immediate         | none                                | the address field itself (a constant) |
| Direct            | EA = address field                  | M[address field]                      |
| Indirect          | EA = M[address field]               | M[M[address field]]                   |
| Register (direct) | none                                | contents of the named register        |
| Register indirect | EA = contents of the register       | M[register]                           |
| Relative          | EA = PC + address field             | M[PC + address field]                 |
| Indexed           | EA = index register + address field | M[index + address field]              |

### WORKED EXAMPLE

PQ 25/26 Q1E

Given PC = 7000, R1 = 1500, address field = 1200, and the memory below, find the effective address and operand for each mode.

| ADDRESS | 7000 | 1200 | 1500 | 3200 | 4500 | 6000 | 8100 |
|---------|------|------|------|------|------|------|------|
| VALUE   | 1200 | 4500 | 6000 | 8100 | 999  | 200  | 400  |

| MODE                   | EFFECTIVE ADDRESS | OPERAND                           |
|------------------------|-------------------|-----------------------------------|
| Immediate              | none              | 1200 (the field is the value)     |
| Direct                 | 1200              | M[1200] = 4500                    |
| Indirect               | M[1200] = 4500    | M[4500] = 999                     |
| Register (R1)          | none              | contents of R1 = 1500             |
| Register indirect (R1) | R1 = 1500         | M[1500] = 6000                    |
| Relative               | PC + 1200 = 8200  | M[8200] (outside the given table) |

**Now redo it:** immediate uses the field as the value; direct reads memory once; indirect reads it twice; register indirect uses the register as a pointer; relative adds the PC. That single sentence answers the whole question.

### WORKED EXAMPLE

SLIDE: INDIRECT FOR READ, ADD AND STORE

Indirect mode works for writing and read-modify, not only for reading. The address field points to a location whose **contents** are the effective address.

- 1 ADD R1, (150)** with R1 = 10, M[150] = 250, M[250] = 5. EA = M[150] = 250; operand = M[250] = 5; so R1 = 10 + 5 = **15**.
- 2 STORE R3, (500)** with R3 = 30, M[500] = 800. EA = M[500] = 800, so the value 30 is written there: M[800] = **30**.

ADD R1, (150) leaves R1 = 15; STORE R3, (500) sets M[800] = 30. Indirect always takes the address from memory first, then reads or writes the operand at it.

Instructions fall into three families, and naming them with examples is a common short question:

- ▶ **Data transfer:** copy data, leaving the source unchanged; only the destination changes. **LOAD** copies from memory into a register, **STORE** copies a register back to memory. **MOVE**, **PUSH**, **POP** move it around too.
- ▶ **Arithmetic and logic:** compute. **ADD**, **SUB**, **MUL**, **DIV**, **AND**, **OR**, **NOT**, shifts.
- ▶ **Control (test and branch):** change what runs next. **JUMP**, **BEQ** (branch if equal), **CALL**, **RETURN**.

Test-and-branch instructions decide what runs next by reading the **condition code register** (also called the flags or status register). An arithmetic or logic operation sets flags as a side effect: **Z** (the result was zero), **N** (negative), **C** (a carry out), **V** (signed overflow). A conditional branch then tests a flag: **BEQ** branches if equal (**Z** set), **BNE** if not equal, **BMI** if minus (**N** set), **BCC** if carry clear, and **BRA** branches always.

Branches are usually **relative**: the target is the PC plus a signed offset, so a **negative offset branches backwards**, which is how loops are built. For example **BNE -40** with PC = 1000 gives EA = 1000 - 40 = **960**, jumping back 40 locations.

Write an assembly program to add two numbers and store the result.

On a one-address (accumulator) machine, load the first, add the second, store the sum:

```
LOAD  A      ; accumulator = A
ADD   B      ; accumulator = A + B
STORE C      ; C = A + B
```

On a three-address machine it is a single instruction: **ADD C, A, B**. Either is a full-mark answer; state which machine you assume.

Do not confuse **indirect** with **register indirect**. Plain indirect goes to memory twice (the field points to a location that holds the real address). Register indirect uses a **register** as the pointer, so it touches memory once. And **immediate** has no effective address at all: the operand is the number in the instruction.

## RECALL AND CHECK

**In direct mode the address field is 1200 and  $M[1200] = 4500$ . What are the effective address and the operand? What changes in indirect mode?**

ANSWER

Direct:  $EA = 1200$ , operand =  $M[1200] = 4500$ . Indirect:  $EA = M[1200] = 4500$ , and the operand is  $M[4500]$ , one further memory read.

## FILL IN THE GAP

1. A 0-address machine evaluates expressions using a \_\_\_\_\_, so the expression is first written in \_\_\_\_\_ order.
2. The register holding the address of the next instruction is the \_\_\_\_\_.
3. In immediate mode the operand is \_\_\_\_\_.
4. In relative mode the effective address is \_\_\_\_\_ plus the address field.

ANSWERS

1. stack; postfix (reverse Polish). 2. program counter (PC). 3. the address field itself (a constant). 4. the PC.

## MODULE TWO

# Processor Design and Performance

*Inside the CPU: the parts that fetch, decode and execute, and then the arithmetic of speed. How fast a processor runs is a number you can calculate, and this module turns clock cycles, instruction mixes and pipelines into marks you can bank.*

$$\text{CPU time} = \text{IC} \times \text{CPI} \times T$$

|              |              |               |
|--------------|--------------|---------------|
| instructions | cycles/instr | seconds/cycle |
|--------------|--------------|---------------|

---

speed is arithmetic

## Processor Components

The parts inside the CPU that carry out the fetch, decode, execute cycle, and the buses that connect them to memory. A few of these carry number questions about address size and memory capacity.

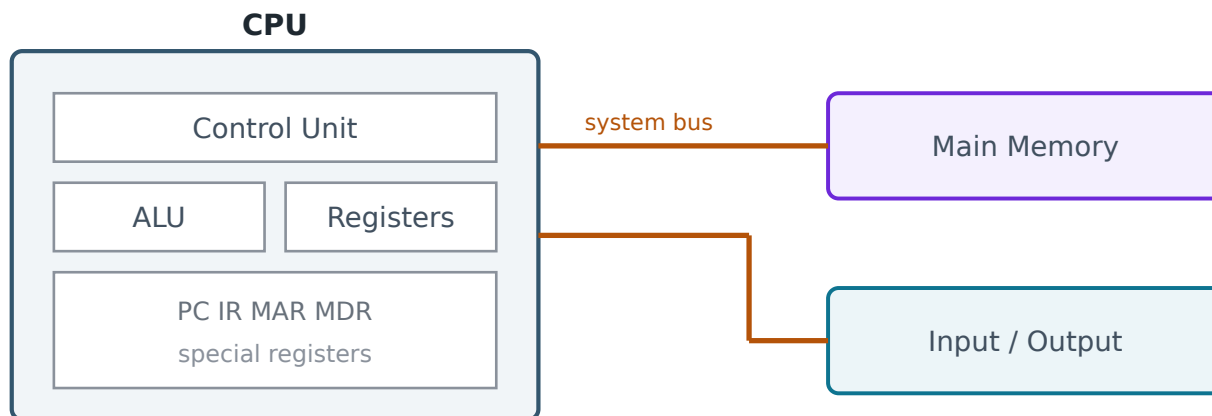
By the end of this unit you can name the components of a CPU and say what each does, describe the three system buses, and calculate address space, memory capacity and address width.

### What is inside the processor

#### TEACH

#### THE CPU'S PARTS

- ▶ **Control Unit (CU):** the conductor. It fetches each instruction, decodes it, and sends the control signals that make every other part act at the right moment.
- ▶ **Arithmetic and Logic Unit (ALU):** the calculator. It performs arithmetic (add, subtract) and logic (AND, OR, NOT, compare, shift).
- ▶ **Registers:** the CPU's own tiny, fastest memory. General-purpose registers hold working values; special ones are the PC, IR, MAR and MDR met in Module One, and the **stack pointer (SP)**, which holds the address of the top of the stack.
- ▶ **Internal buses:** the wires that carry data between the registers, the ALU and the control unit.



*The CPU, memory and I/O joined by the system bus.*

#### TEACH

#### THE THREE SYSTEM BUSES

The **system bus** that joins CPU, memory and I/O is really three sets of wires:

- ▶ **Address bus:** carries the address the CPU wants to read or write. Its width sets how many locations can be addressed: **k address lines reach  $2^k$  locations**. One direction only (CPU to memory).
- ▶ **Data bus:** carries the actual data, both directions. Its width sets how many bits move per transfer.
- ▶ **Control bus:** carries the signals that say read, write, or wait.

The third partner on the system bus is **input/output (I/O)**: the devices that move data between the computer and the outside world (keyboard, screen, disk, network). The CPU manages I/O in one of three ways:

- ▶ **Programmed I/O (polling)**: the CPU repeatedly checks whether a device is ready. Simple, but it wastes CPU time waiting.
- ▶ **Interrupt-driven I/O**: the device raises an **interrupt** when it is ready and the CPU stops to service it, so it need not poll.
- ▶ **Direct memory access (DMA)**: a DMA controller moves a whole block between a device and memory without the CPU, interrupting it only once the transfer is done. Fastest for large transfers.

## ● MUST MEMORISE

## ADDRESSING CAPACITY

**k address lines can address  $2^k$  locations.** Turn it around: to address N locations you need  **$k = \log_2 N$**  address lines. The total capacity is  $2^k \times (\text{bytes per location})$ .

**Memory hook:** every extra address line **doubles** the number of locations you can reach.

## WORKED EXAMPLE

## PQ 20/21 Q1A

A minicomputer uses **18 address lines**. Find (i) the address space, (ii) the largest memory in bytes if each location holds one byte, (iii) the length of the program counter.

- 1 Address space =  $2^{\text{number of lines}} = 2^{18} = 262\,144$  locations.  
 $2^{18} = 2^{10} \times 2^8 = 1024 \times 256$ .
- 2 At one byte per location that is  $262\,144$  bytes = 256 KB.
- 3 The PC must hold any address, so it is as wide as the address bus: 18 bits.

(i)  $2^{18} = 262\,144$  locations. (ii)  $262\,144$  bytes = 256 KB. (iii) PC length = 18 bits.

**Now redo it:** address space is always 2 to the power of the number of address lines, and the PC is always as wide as an address.

A computer has **4 GB** of word-addressable memory with a word size of **2 bytes**. How many bits are needed to address memory?

- 1**  $4 \text{ GB} = 4 \times 2^{30} \text{ bytes} = 2^{32} \text{ bytes}$ .
- 2** Word-addressable means each address names a word, not a byte. Number of words  $= 2^{32} \text{ bytes} \div 2 \text{ bytes} = 2^{31} \text{ words}$ .
- 3** To address  $2^{31}$  words needs **31** bits (because  $2^{31}$  words need 31 address lines).

$4 \text{ GB} \div 2\text{-byte word} = 2^{31} \text{ words}$ , so **31 address bits** are needed.

**Now redo it:** the trap is word-addressable versus byte-addressable. Divide the total bytes by the bytes per addressable unit first, then take the log base 2.

## RECALL AND CHECK

**A machine has 20 address lines and a 1-byte cell. How much memory can it address?**

ANSWER

$2^{20} = 1\,048\,576 \text{ locations} = 1 \text{ MB}$ . Every address line doubles the reach, and  $2^{20}$  is exactly one mega.

## MODULE TWO · UNIT 2

# CPU Performance Metrics

*How fast a processor runs is not a mystery, it is a calculation. This is the most heavily weighted numeric topic on the recent papers, so every formula here is worth learning to reflex.*

*By the end of this unit you can compute CPU execution time, average cycles per instruction from a mix, MIPS, the effect of an optimisation, and overall speedup by Amdahl's law. The favourite exam questions of this type are worked in full.*

## The one performance equation

A program takes time equal to how many instructions it runs, times how many clock cycles each takes on average, times how long one clock cycle lasts. That is the whole of it.

**CPU time** =  $IC \times CPI \times T$ , where IC is the instruction count, CPI is the average cycles per instruction, and T is the clock cycle time (seconds per cycle). Since  $T = 1 / \text{clock rate}$ :

- ▶ **CPU time** =  $(IC \times CPI) / \text{clock rate}$
- ▶ **Average CPI** =  $\sum (\text{fraction of type } i \times \text{CPI of type } i)$
- ▶ **MIPS** =  $\text{clock rate} / (CPI \times 10^6)$  (millions of instructions per second)
- ▶ **Speedup** =  $\text{time}(\text{old}) / \text{time}(\text{new})$ ; **improvement** =  $(\text{old} - \text{new}) / \text{old}$

**Memory hook:** "I see PT" for  $IC \times CPI \times T$ . Fewer instructions, fewer cycles each, or a shorter cycle: any of the three cuts the time.

## WORKED EXAMPLE

PQ 25/26 Q1D

A processor runs **1 million** instructions with this mix of addressing modes and cycle costs. (i) Average CPI. (ii) Total clock cycles. (iii) New CPI if indirect is optimised to 2 cycles. (iv) The performance improvement.

| MODE   | REGISTER | IMMEDIATE | DIRECT | INDIRECT | INDEXED | RELATIVE |
|--------|----------|-----------|--------|----------|---------|----------|
| SHARE  | 40%      | 20%       | 15%    | 10%      | 10%     | 5%       |
| CYCLES | 1        | 1         | 2      | 4        | 3       | 2        |

- 1 Average CPI = weighted sum:  $0.40(1) + 0.20(1) + 0.15(2) + 0.10(4) + 0.10(3) + 0.05(2)$ .  
 $= 0.40 + 0.20 + 0.30 + 0.40 + 0.30 + 0.10$ .
- 2 Average CPI = **1.70**.
- 3 Total clock cycles =  $IC \times CPI = 1\,000\,000 \times 1.70 = 1\,700\,000$  cycles.
- 4 Optimise indirect from 4 to 2 cycles: its contribution falls from  $0.10(4) = 0.40$  to  $0.10(2) = 0.20$ , a drop of 0.20. New CPI =  $1.70 - 0.20 = \mathbf{1.50}$ .
- 5 Improvement =  $(1.70 - 1.50) / 1.70 = 0.20 / 1.70 = \mathbf{11.76\%}$  faster (equivalently a speedup of  $1.70 / 1.50 = 1.133$  times).

(i) CPI = 1.70. (ii) 1 700 000 cycles. (iii) New CPI = 1.50. (iv) Improvement = 11.76% (speedup 1.13 times).

**Now redo it:** average CPI is just a weighted average. To measure an optimisation, recompute CPI with the one changed cost and compare. This is the cleanest marks on the paper.

## When different instructions take different numbers of cycles

The same weighted-average idea handles a pipeline where some instruction types add penalty cycles. Start from an ideal CPI of 1 (one instruction finished per cycle) and add the average extra cycles each type contributes.



A 5-stage pipeline (IF fetch, ID decode, EX execute, MEM memory, WB write-back) has stage times IF 5, ID 4, EX 6, MEM 5, WB 4 (ns) and 1 ns register overhead. Instruction mix: Arithmetic 40%, Memory 35%, Branch 20%, Floating point 5%, over 20 million instructions. The non-pipelined clock is 25 ns. Find the pipelined clock, the actual CPI, the execution time, and the speedup.

**Reading the penalties (state your assumptions, the wording is dense):** ideal CPI 1; branch adds 2 extra cycles (EX takes 3); a memory access adds a 2-cycle penalty plus, on a 10% cache miss, 4 more (so  $2 + 0.10 \times 4 = 2.4$  average); floating point adds 4 extra; arithmetic adds none.

- 1 Pipelined clock = slowest stage + overhead =  $6 + 1 = 7$  ns.  
*A pipeline runs at the speed of its slowest stage.*
- 2 Actual CPI =  $1 + 0.20(2) + 0.35(2.4) + 0.05(4) + 0.40(0) = 1 + 0.40 + 0.84 + 0.20 = 2.44$ .
- 3 Execution time =  $IC \times CPI \times T = 20\,000\,000 \times 2.44 \times 7 \text{ ns} = 341.6 \text{ ms}$  (0.3416 s).
- 4 Non-pipelined time =  $20\,000\,000 \times 25 \text{ ns} = 500 \text{ ms}$  (0.5 s).
- 5 Speedup =  $0.5 / 0.3416 = 1.46$  times.

Clock = 7 ns; CPI = 2.44; execution time = 0.3416 s; speedup  $\approx 1.46$  (under the stated reading of the penalties).

**Now redo it:** the method never changes even if you read a penalty differently: clock = slowest stage + overhead, CPI =  $1 + \text{weighted extra cycles}$ , time =  $IC \times CPI \times T$ . State how you read each penalty and you keep the method marks.

## Amdahl's law: the limit on any speedup

If you speed up only part of a task, the untouched part limits the gain. Amdahl's law puts a number on it, and the exam asks it most years.

### ● MUST MEMORISE

### AMDAHL'S LAW

If a fraction **f** of the work is sped up by a factor **s**, the overall speedup is

$$\text{Speedup} = 1 / ( (1 - f) + f / s )$$

**Memory hook:** the part you did not speed up,  $(1 - f)$ , never goes away, so it caps the total gain no matter how large **s** is.

A cache is 10 times faster than main memory, and the CPU finds what it needs in the cache **85%** of the time. Find the overall speedup by Amdahl's law.

- 1 The sped-up fraction is  $f = 0.85$  (the cache accesses), and the speedup factor is  $s = 10$ .
- 2  $\text{Speedup} = 1 / ((1 - 0.85) + 0.85 / 10) = 1 / (0.15 + 0.085)$ .
- 3  $= 1 / 0.235 = \mathbf{4.26}$ .

Speedup =  $1 / (0.15 + 0.085) = 1 / 0.235 \approx \mathbf{4.26 \text{ times}}$ .

**Now redo it:** notice the answer is far below 10, because the 15% of slow accesses dominate. That is exactly the point Amdahl's law makes.

### ⚠ TRAP

Two slips. (1) Improvement percentage is measured against the **old** time,  $(\text{old} - \text{new}) / \text{old}$ , not against the new one. (2) In Amdahl's law, **f** is the fraction of work that is sped up, and you divide only that part by  $s$ ; the rest,  $(1 - f)$ , stays at full cost.

### RECALL AND CHECK

**A program runs 5 million instructions at an average CPI of 2 on a 1 GHz clock. How long does it take?**

ANSWER

CPU time =  $\text{IC} \times \text{CPI} / \text{clock rate} = (5\,000\,000 \times 2) / 10^9 = 10\,000\,000 / 10^9 = 0.01 \text{ s} = 10 \text{ ms}$ .

### FILL IN THE GAP

1. CPU time = instruction count  $\times$  \_\_\_\_\_  $\times$  clock cycle time.
2. Average CPI is the \_\_\_\_\_ average of each type's cycle cost.
3. A pipeline's clock cycle equals its \_\_\_\_\_ stage time plus the register overhead.
4. In Amdahl's law the term  $(1 - f)$  represents the part that is \_\_\_\_\_.

ANSWERS

1. CPI. 2. weighted. 3. slowest (longest). 4. not sped up.

## MODULE TWO · UNIT 3

# Pipelining

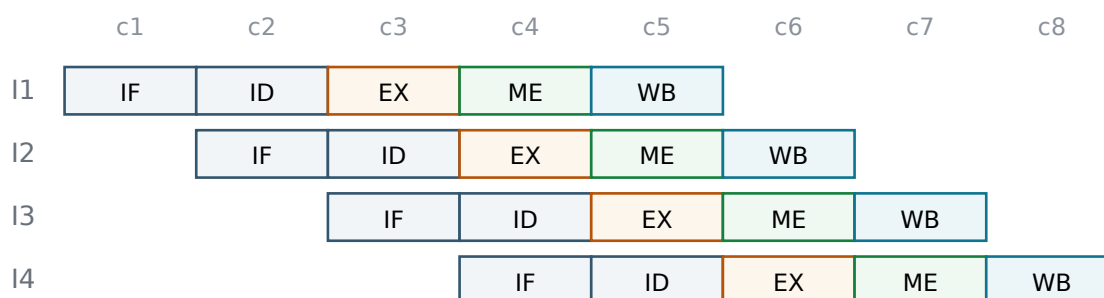
*Overlapping instructions so several are in progress at once, the way a laundry overlaps washing, drying and folding. It speeds a processor up, and it comes with hazards you must handle. Both the speedup sums and the hazard diagrams are exam favourites.*

*By the end of this unit you can compute a pipeline's cycle time, its time for  $n$  tasks, its speedup and throughput; identify structural, data and control hazards; and draw a timing diagram showing*

stalls and forwarding.

## The idea: overlap the stages

Split each instruction into  $k$  stages. Without a pipeline, one instruction finishes before the next begins. With a pipeline, as soon as an instruction leaves stage 1 the next enters it, so after the pipeline fills, **one instruction completes every clock cycle**. The clock cycle is set by the slowest stage plus the small latch delay between stages.



*Four instructions overlapped in a 5-stage pipeline. After it fills, one finishes each cycle.*

### ● MUST MEMORISE

### PIPELINE FORMULAS

- ▶ **Cycle time** = slowest stage time + latch delay.
- ▶ **Time for  $n$  tasks** =  $(k + n - 1) \times \text{cycle time}$ , for  $k$  stages.
- ▶ **Non-pipelined time** =  $n \times (\text{sum of all stage times})$ .
- ▶ **Speedup** = non-pipelined time / pipelined time. Its ceiling is  $k$ .
- ▶ **Throughput** =  $n / \text{pipelined time}$  (tasks per second); in the steady state this is  $1 / \text{cycle time}$ .

**Memory hook:** " $k + n - 1$ " is  $k$  cycles to fill the pipe with the first task, then one more cycle for each of the remaining  $n - 1$  tasks.

A pipeline has 4 phases of duration 60, 50, 90 and 80 ns, with a 10 ns latch delay. Find (i) the cycle time, (ii) the non-pipeline time for one task, (iii) the speedup ratio, (iv) the pipeline time for 500 tasks, (v) the sequential time for 500 tasks, (vi) the throughput.

- 1 Cycle time = slowest phase + latch =  $90 + 10 = 100$  ns.
- 2 Non-pipeline time for one task =  $60 + 50 + 90 + 80 = 280$  ns.
- 3 Speedup ratio (ideal) =  $280 / 100 = 2.8$ .
- 4 Pipeline time for 500 tasks =  $(k + n - 1) \times \text{cycle} = (4 + 499) \times 100 = 503 \times 100 = 50\,300$  ns.
- 5 Sequential time for 500 tasks =  $500 \times 280 = 140\,000$  ns.
- 6 Throughput =  $500 / 50\,300 \text{ ns} \approx 9.94$  million tasks per second (in the steady state,  $1 / 100 \text{ ns} = 10$  million per second). Actual speedup for 500 tasks =  $140\,000 / 50\,300 = 2.78$ .

Cycle 100 ns; non-pipe 280 ns; ideal speedup 2.8; pipe(500) = 50 300 ns; seq(500) = 140 000 ns; throughput  $\approx 9.94$  M tasks/s.

**Now redo it:** cycle = slowest + latch, pipe =  $(k + n - 1) \times \text{cycle}$ , and everything else falls out of those two.

Four operations run through a 5-stage pipeline with stage times 8, 9, 11, 13 and 16 ns. Compare the non-pipelined and pipelined execution times and give the speedup.

- 1 Non-pipelined: each operation passes through all stages,  $8+9+11+13+16 = 57$  ns each, so  $4 \times 57 = 228$  ns.
- 2 Pipelined cycle time = slowest stage = 16 ns.
- 3 Pipelined time =  $(k + n - 1) \times \text{cycle} = (5 + 4 - 1) \times 16 = 8 \times 16 = 128$  ns.
- 4 Speedup =  $228 / 128 = 1.78$ .

Non-pipelined 228 ns, pipelined 128 ns, speedup  $\approx 1.78$ . The speedup is below 5 because only 4 tasks run, so the fill time still dominates.

## Hazards: when overlap goes wrong

Overlapping instructions can clash. There are three kinds of hazard, and the exam expects you to name them and say how each is handled.

- ▶ **Structural hazard:** two instructions need the same hardware at once (for example one memory for both a fetch and a data access). Fixed by duplicating the resource, such as separate instruction and data caches.
- ▶ **Data hazard:** an instruction needs a result the previous one has not written back yet (a read-after-write, RAW). Reduced by **forwarding** (feeding the result straight from where it is produced to where it is needed) and, when that is not enough, by **stalling** one or more cycles.
- ▶ **Control hazard:** a branch changes what runs next, but the pipeline has already fetched the following instructions. Handled by resolving the branch early, predicting it, or flushing the wrongly fetched instructions.

The one data hazard forwarding cannot fully hide is **load-use**: a value loaded from memory is not ready until the memory stage, so an instruction that uses it the very next cycle must wait one cycle even with forwarding. That single unavoidable stall appears in the exam's RISC pipeline question.

A 5-stage RISC (reduced instruction set) pipeline (IF, ID, EX, MEM, WB) runs the code below, with forwarding available, a one-cycle stall for a load-use hazard, and a multiplier that occupies EX for three cycles. Identify the hazards, draw the timing diagram, and give the total cycles.

```
I1: LW   R1, 0(R2)    ; load R1 from memory
I2: ADD  R3, R1, R4    ; needs R1  -> load-use hazard with I1
I3: SUB  R5, R3, R6    ; needs R3  (forwarded from I2)
I4: SW   R5, 8(R2)    ; needs R5  (forwarded from I3)
I5: BEQ  R5, R0, LABEL ; branch, resolved in EX
I6: MUL  R7, R5, R8    ; multiply, EX takes 3 cycles
```

### Hazards

I1 to I2 is a **load-use data hazard**: R1 is not ready until I1's MEM stage, so I2 stalls one cycle. I2 to I3, I3 to I4, I3 to I5 and I3 to I6 are ordinary data hazards **hidden by forwarding**. I6's **multiply holds EX for three cycles**, adding two extra cycles at the end (assuming the branch is not taken).

Timing diagram (each cell is the stage that instruction is in)

| CYCLE  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 |
|--------|----|----|----|----|----|----|----|----|----|----|----|----|----|
| I1 LW  | IF | ID | EX | ME | WB |    |    |    |    |    |    |    |    |
| I2 ADD |    | IF | ID | -- | EX | ME | WB |    |    |    |    |    |    |
| I3 SUB |    |    | IF | -- | ID | EX | ME | WB |    |    |    |    |    |
| I4 SW  |    |    |    |    | IF | ID | EX | ME | WB |    |    |    |    |
| I5 BEQ |    |    |    |    |    | IF | ID | EX | ME | WB |    |    |    |
| I6 MUL |    |    |    |    |    |    | IF | ID | EX | EX | EX | ME | WB |

One load-use stall (the -- in cycle 4) plus two extra EX cycles for the multiply give a total of **13 cycles** (a stall-free run of 6 instructions would take  $5 + 6 - 1 = 10$ ).

Two hardware improvements that reduce stalls

Add **forwarding paths** (already assumed here) and a **pipelined or duplicated multiplier** so a long multiply does not block EX; separate instruction and data memories remove structural stalls.

**Now redo it:** mark the one load-use stall, let forwarding cover the rest, and remember a 3-cycle EX adds two cycles. Count the last WB: that cycle number is the total.

## Multi-cycle execute, with and without forwarding

When some instructions hold the EX stage for several cycles, they stall the ones behind them, and a dependent instruction must wait for its operand. The next question asks you to schedule exactly this, both with forwarding and without.

A 5-stage pipeline (IF, ID, OF, EX, WB) takes 1 cycle per stage, except EX takes 1 cycle for ADD and SUB, 3 for MUL and 6 for DIV. Schedule this sequence and give the total cycles, with and without operand forwarding.

I1: MUL R2, R0, R1      I2: DIV R5, R3, R4  
I3: ADD R2, R5, R2      I4: SUB R5, R2, R6

I3 needs R5 from I2 and R2 from I1; I4 needs R2 from I3.

Stated assumptions

One execution unit, in order: an instruction enters EX only when EX is free, so a long EX stalls those behind it (a stalled instruction repeats its stage below). Write-back is one cycle after EX. **With forwarding**, a result reaches a dependent instruction's EX the cycle after it is produced; **without forwarding**, the dependent must wait until the producer has written back.

(i) With operand forwarding

| CYCLE | I1 MUL | I2 DIV | I3 ADD | I4 SUB |
|-------|--------|--------|--------|--------|
| 1     | IF     |        |        |        |
| 2     | ID     | IF     |        |        |
| 3     | OF     | ID     | IF     |        |
| 4     | EX     | OF     | ID     | IF     |
| 5     | EX     | OF     | ID     | IF     |
| 6     | EX     | OF     | ID     | IF     |
| 7     | WB     | EX     | OF     | ID     |
| 8     |        | EX     | OF     | ID     |
| 9     |        | EX     | OF     | ID     |
| 10    |        | EX     | OF     | ID     |
| 11    |        | EX     | OF     | ID     |
| 12    |        | EX     | OF     | ID     |
| 13    |        | WB     | EX     | OF     |
| 14    |        |        | WB     | EX     |
| 15    |        |        |        | WB     |

The DIV holds EX for cycles 7 to 12; I3 forwards R5 straight from it into EX at cycle 13, and I4 forwards R2 from I3 at cycle 14. Total = **15 cycles**.

(ii) Without operand forwarding

Now I3 cannot read R5 until I2 has written back (cycle 13), so its EX slips to cycle 14; and I4 must wait for I3's write-back (cycle 15), so its EX slips to cycle 16.

| CYCLE | I1 MUL | I2 DIV | I3 ADD | I4 SUB |
|-------|--------|--------|--------|--------|
| 7     | WB     | EX     | OF     | ID     |
| 12    |        | EX     | OF     | ID     |
| 13    |        | WB     | OF     | ID     |

|    |    |    |
|----|----|----|
| 14 | EX | 0F |
| 15 | WB | 0F |
| 16 |    | EX |
| 17 |    | WB |

I1 to I2 run exactly as above (cycles 1 to 13). Then I3 waits for I2's write-back and executes at cycle 14, and I4 waits for I3's and executes at cycle 16. Total = **17 cycles**. Forwarding saved 2 cycles here; it saves more when short instructions depend on each other back to back.

**Now redo it:** lay out the long DIV first (it fixes everything behind it), then place each dependent instruction's EX one cycle after its operand is ready: after the producer's EX with forwarding, after its write-back without.

#### ⚠ TRAP

Forwarding removes **most** data-hazard stalls but not the **load-use** one: a loaded value simply is not available in time for the very next instruction, so exactly one stall remains. And do not forget that a multi-cycle execute (multiply, divide) holds the EX stage and delays everything behind it.

#### RECALL AND CHECK

**A 5-stage pipeline runs 100 independent instructions with no hazards. How many cycles, and what is the speedup over the non-pipelined version?**

ANSWER

Cycles =  $k + n - 1 = 5 + 100 - 1 = 104$ . The non-pipelined version takes  $100 \times 5 = 500$  stage-times, so the speedup is  $500 / 104 \approx 4.8$ , close to the ceiling of 5.

#### FILL IN THE GAP

1. A k-stage pipeline finishes n tasks in \_\_\_\_\_ cycles.
2. The pipeline cycle time is set by the \_\_\_\_\_ stage plus the latch delay.
3. A read-after-write clash is a \_\_\_\_\_ hazard, eased by \_\_\_\_\_.
4. The one stall forwarding cannot remove is the \_\_\_\_\_ hazard.

ANSWERS

1.  $k + n - 1$ . 2. slowest (longest). 3. data (RAW); forwarding. 4. load-use.



## MODULE THREE

# Computer Memory Organization

*Memory is a hierarchy: a little fast storage close to the processor, a lot of slow storage far away, and the art of keeping the data you need in the fast part. Cache is where this module earns its marks, and cache is all over the recent papers.*



# Computer Memory Overview

*Why memory is built as a hierarchy, how memory is accessed, and the principle of locality that makes the whole scheme work.*

*By the end of this unit you can describe the memory hierarchy and the four access methods, and explain locality of reference, including temporal and spatial locality. The recurring exam questions of this type are answered here.*

## The memory hierarchy

No single memory is both fast and large and cheap. So computers use several, arranged in a hierarchy: fast, small, expensive memory near the CPU, backed by slower, larger, cheaper memory further away. Data moves up the hierarchy when it is needed and down when it is not.

### ● MUST MEMORISE

### THE HIERARCHY

From fastest and smallest to slowest and largest: **registers, cache (L1, L2, L3), main memory (RAM), secondary storage (disk, SSD), tertiary (tape).**

**Memory hook:** as you go down, everything grows except speed and cost per byte: **bigger, slower, cheaper.** As you go up: **smaller, faster, dearer.**

### TEACH

### ACCESS METHODS

- ▶ **Sequential:** read from the start in order until you reach the item (magnetic tape). Slow, access time depends on position.
- ▶ **Direct:** jump near the item, then search a small region (disk: seek the track, then read round it).
- ▶ **Random:** any location takes the same time to reach, by its address (RAM).
- ▶ **Associative:** located by content, not address; all locations checked at once (used inside cache to find a tag). Fastest, most expensive.

### TEACH

### CHARACTERISTICS AND VOCABULARY OF MEMORY

Memory is described by four **characteristics**: its **location** (registers, cache, main, external), its **access method** (above), its **performance** (access time and transfer rate), and its **capacity**. The units of capacity, smallest first:

- ▶ **Bit**, one 0 or 1. A **nibble** is 4 bits; a **byte** is 8 bits.
- ▶ **Cell**, one addressable location (often a byte); an **address** names one cell.
- ▶ **Word**, the natural unit the CPU works on (say 32 or 64 bits).
- ▶ **Block**, the unit moved between levels (several words), the key idea for cache.

If a memory holds  $2^n$  words with  $K$  words per block, it has  $M = 2^n / K$  blocks, and needs an **n-bit address**.

How many bits does a **4 MB** byte-addressable memory hold, and how many address lines does it need?

- 1 4 MB =  $4 \times 1024 \times 1024 = 4\,194\,304$  bytes.
- 2 Bits:  $4\,194\,304 \times 8 = 33\,554\,432$  bits  $\approx$  **33.5 million bits**.
- 3 Address lines:  $4\,194\,304 = 2^{22}$  cells, so **22** address lines.

A 4 MB memory = 4 194 304 bytes = 33 554 432 bits ( $\approx$  33.5 Mbit), addressed by 22 lines ( $2^{22} = 4\,194\,304$ ).

Cache works because real programs do not touch memory randomly. They obey the **principle of locality of reference**: over any short period, memory accesses cluster in a small area. It has two forms:

- ▶ **Temporal locality**: a location just used is likely to be used again soon (a loop counter, a variable). So keep recently used data close.
- ▶ **Spatial locality**: a location near one just used is likely to be used soon (the next array element, the next instruction). So fetch neighbours together, a whole block at a time.

These are exactly why cache stores **recently used blocks** (temporal) and stores data in **blocks of neighbours** (spatial).

Explain locality of reference, and distinguish temporal from spatial locality.

Locality of reference is the tendency of a program to access a small, predictable set of memory locations during any short period. Temporal locality: a location accessed now is likely to be accessed again soon (so cache keeps recently used items). Spatial locality: locations near one just accessed are likely to be accessed soon (so cache fetches a whole block of neighbouring words at once).

A two-level store has a fast level (1000 words, access 0.01  $\mu$ s) and a slow level (100 000 words, access 0.1  $\mu$ s). If 95% of accesses are found in the fast level, what is the average access time?

- 1 Hit (95%): the word is in the fast level, costing 0.01  $\mu$ s.
- 2 Miss (5%): pay the fast level, then the slow level:  $0.01 + 0.1 = 0.11$   $\mu$ s.
- 3 Average =  $0.95 \times 0.01 + 0.05 \times 0.11 = 0.0095 + 0.0055 = \mathbf{0.015}$   $\mu$ s.

Average access time =  $0.95(0.01) + 0.05(0.11) = \mathbf{0.015}$   $\mu$ s, close to the fast level's 0.01  $\mu$ s and far below the slow level's 0.1  $\mu$ s. That gap, bought by a high hit rate, is the whole reason for the hierarchy. The same formula scales to any two levels (see average memory access time, AMAT, in Unit 3).

## RECALL AND CHECK

**A loop repeatedly reads the same counter and steps through an array. Which kind of locality does each show?**

ANSWER

The repeated counter is **temporal** locality (same location, used again and again). Stepping through the array is **spatial** locality (successive nearby locations).

## MODULE THREE · UNIT 2

## Semiconductor Memory

*The chips that make up main memory: read-write RAM and read-only ROM, and the two kinds of RAM the exam always compares, DRAM and SRAM.*

*By the end of this unit you can distinguish RAM from ROM and give four differences between DRAM and SRAM, the exact question that recurs on the paper.*

## TEACH

## RAM AND ROM

**RAM (Random Access Memory)** is read-write and **volatile**: it loses its contents when power is removed. It holds the running program and its data. **ROM (Read Only Memory)** is read-mostly and **non-volatile**: it keeps its contents without power, and holds firmware such as the boot code. Both are random access: any location takes the same time to reach.

## TEACH

## DRAM AND SRAM

RAM comes in two technologies. **Dynamic RAM (DRAM)** stores each bit as a charge on a tiny capacitor, which leaks, so it must be **refreshed** thousands of times a second. **Static RAM (SRAM)** stores each bit in a latch of several transistors, which holds as long as power is on, so it needs no refresh. SRAM is faster and simpler to use but larger and dearer per bit, so it is used for cache; DRAM is denser and cheaper, so it is used for main memory.

State four differences between DRAM and SRAM.

| DYNAMIC RAM (DRAM)                    | STATIC RAM (SRAM)                  |
|---------------------------------------|------------------------------------|
| Stores a bit as charge on a capacitor | Stores a bit in a transistor latch |
| Must be refreshed periodically        | Needs no refresh while powered     |
| Slower access                         | Faster access                      |
| Higher density, cheaper per bit       | Lower density, dearer per bit      |
| Used for main memory                  | Used for cache memory              |

**Now redo it:** the four you must have are refresh, speed, density and cost. The memory hook: **DRAM** is **D**ense, **D**ynamic (needs refresh) and cheap; **SRAM** is **S**tatic (no refresh), **S**peedy and costly.

## RECALL AND CHECK

**Why is SRAM used for cache and DRAM for main memory?**

ANSWER

SRAM is faster and needs no refresh, which suits the small, speed-critical cache; DRAM is denser and cheaper per bit, which suits large main memory where cost per bit matters more than raw speed.

## MODULE THREE · UNIT 3

# Cache Memory Design

*A small fast memory that holds copies of the data most likely to be needed, so the CPU rarely waits on slow main memory. Splitting an address into cache fields and finding a block are the most examined calculations on the whole course.*

*By the end of this unit you can define hit, miss and hit ratio; compute average memory access time; break an address into tag, index and offset for direct, associative and set-associative mapping; and find which cache block an address maps to. Every recent cache question is worked here.*

## Why cache works, and the vocabulary

Because of locality (Unit 1), a small store of recently used blocks satisfies most accesses. When the CPU needs data, it checks cache first. A **hit** means it is there; a **miss** means it must be fetched from main memory (and the whole **block** around it is brought in, for spatial locality).

**Hit ratio  $h$**  = hits / total accesses; miss ratio =  $1 - h$ .

**Average memory access time** depends on how cache and memory are arranged:

- ▶ **Hierarchical (look-through):** always check cache first, go to memory only on a miss.  $AMAT = T_{\text{cache}} + (1 - h) \times T_{\text{memory}}$ .
- ▶ **Simultaneous (look-aside):** start cache and memory together, use whichever is right.  $AMAT = h \times T_{\text{cache}} + (1 - h) \times T_{\text{memory}}$ .

**Memory hook:** a high hit ratio pulls the average toward the fast cache time; even a small miss rate hurts, because a miss pays the full slow memory cost.

## Splitting the address: tag, index, offset

Main memory is divided into blocks; cache is divided into lines the same size as a block. A memory address is split into fields that say which block, which line, and where inside the block.

### TEACH

### THE THREE MAPPING FUNCTIONS

|                  |     |              |        |
|------------------|-----|--------------|--------|
| <b>Direct</b>    | Tag | Line (index) | Offset |
| <b>Set-assoc</b> | Tag | Set          | Offset |
| <b>Assoc</b>     | Tag |              | Offset |

*All three carry an offset. Direct has a line field, set-associative a set field, fully associative neither.*

- ▶ **Direct mapping:** each block has exactly one line it may sit in,  $\text{line} = (\text{block number}) \bmod (\text{number of lines})$ . Simple and fast, but two blocks that map to the same line keep evicting each other (conflict misses).
- ▶ **Fully associative:** a block may sit in any line. No conflict misses, but every line's tag must be searched, which is expensive.
- ▶ **Set associative (k-way):** the compromise. Lines are grouped into sets of  $k$ ; a block maps to one set,  $\text{set} = (\text{block number}) \bmod (\text{number of sets})$ , and may sit in any of the  $k$  lines in it.

## MUST MEMORISE

## FIELD WIDTHS

Work from the block and cache sizes:

- ▶ **Offset bits** =  $\log_2(\text{bytes, or words, per block})$ .
- ▶ **Number of lines** = cache size / block size.
- ▶ **Index bits** (direct) =  $\log_2(\text{number of lines})$ ; **set bits** (k-way) =  $\log_2(\text{number of sets})$ , where sets = lines / k.
- ▶ **Tag bits** = address bits – index (or set) bits – offset bits.

**Memory hook:** offset picks the byte in the block, index/set picks the slot, tag is everything left over to confirm the right block is there.

## MUST MEMORISE

## TAG DIRECTORY SIZE

Each cache line stores a **tag** beside its data, to prove which block it currently holds. The storage spent on tags, the **tag directory**, is **number of lines × tag bits per line**. It grows as the mapping gets more associative, because dropping the index (or shrinking the set field) lengthens the tag on every line.

## WORKED EXAMPLE

## PQ 25/26 Q2A

A computer has 32-bit addresses. Each block holds 16 words and a direct-mapped cache has 256 blocks. In which cache line does each hex address fall?

- 1 Field widths: offset =  $\log_2 16 = 4$  bits; index =  $\log_2 256 = 8$  bits; tag =  $32 - 8 - 4 = 20$  bits.
- 2 The shortcut in hex: the **last hex digit** is the 4-bit offset, and the **next two hex digits** are the 8-bit line (index). Read them straight off.

| ADDRESS  | OFFSET (LAST DIGIT) | LINE (NEXT TWO DIGITS) | LINE IN DECIMAL |
|----------|---------------------|------------------------|-----------------|
| CA2DFC12 | 2                   | C1                     | 193             |
| F24F25FF | F                   | 5F                     | 95              |
| 24BBABCD | D                   | BC                     | 188             |
| C1D9EEF2 | 2                   | EF                     | 239             |

Lines: CA2DFC12 → 193, F24F25FF → 95, 24BBABCD → 188, C1D9EEF2 → 239.

**Now redo it:** once you know offset is 4 bits and index is 8 bits, the hex reads straight off: drop one digit, take the next two. Confirm the method on the tutorial address 1A2BC012, which gives line 1.

Main memory is 64 MB, the cache is 128 KB, and a block is 64 bytes. Find the number of cache lines, then the field widths for direct and 4-way set associative mapping.

- 1 Address bits:  $64 \text{ MB} = 2^{26}$  bytes, so **26-bit** addresses.
- 2 Number of lines = cache / block =  $128 \text{ KB} / 64 \text{ B} = 2^{17} / 2^6 = 2^{11} = \mathbf{2048 \text{ lines}}$ .
- 3 Offset =  $\log_2 64 = 6$  bits.
- 4 Direct: index =  $\log_2 2048 = 11$  bits, so tag =  $26 - 11 - 6 = \mathbf{9 \text{ bits}}$ .
- 5 4-way: sets =  $2048 / 4 = 512$ , so set bits =  $\log_2 512 = 9$ , and tag =  $26 - 9 - 6 = \mathbf{11 \text{ bits}}$ .

2048 lines. Direct: tag 9, index 11, offset 6. 4-way: tag 11, set 9, offset 6.  
(Grouping lines into sets moves bits from the index into the tag.)

A cache is **16 KB**, block **256 bytes**, main memory **128 KB**. Find the address format and the tag-directory size for direct, fully associative, and 2-way set-associative mapping.

- 1 Address:  $128 \text{ KB} = 2^{17}$ , so **17-bit** physical addresses. Offset =  $\log_2 256 = \mathbf{8 \text{ bits}}$ .  
Lines =  $16 \text{ KB} / 256 \text{ B} = \mathbf{64}$ .
- 2 **Direct:** index =  $\log_2 64 = 6$ , tag =  $17 - 6 - 8 = \mathbf{3}$ . Tag directory =  $64 \times 3 = 192$  bits = **24 bytes**.
- 3 **Fully associative:** no index, tag =  $17 - 8 = \mathbf{9}$ . Tag directory =  $64 \times 9 = 576$  bits = **72 bytes**.
- 4 **2-way set associative:** sets =  $64 / 2 = 32$ , set bits =  $\log_2 32 = 5$ , tag =  $17 - 5 - 8 = \mathbf{4}$ . Tag directory =  $64 \times 4 = 256$  bits = **32 bytes**.

Same 64 lines throughout, but the tag directory grows from **24 bytes** (direct) to **32 bytes** (2-way) to **72 bytes** (fully associative), because less indexing leaves a longer tag on every line.



### WORKED EXAMPLE

PQ 25/26 Q2C

Main memory 1 GB, cache 512 KB, block 128 bytes, 4-way set associative. Find (i) the number of cache blocks, (ii) the number of sets, (iii) the address format, (iv) the AMAT if the hit rate is 96%, cache access is 2 ns and main memory is 80 ns.

- 1 Blocks (lines) = cache / block = 512 KB / 128 B =  $2^{19} / 2^7 = \mathbf{4096 \text{ blocks}}$ .
- 2 Sets = blocks / 4 = 4096 / 4 = **1024 sets**.
- 3 Address = 1 GB =  $2^{30}$  so 30 bits. Offset =  $\log_2 128 = 7$ , set =  $\log_2 1024 = 10$ , tag =  $30 - 10 - 7 = 13$ . Format: **tag 13, set 10, offset 7**.
- 4 AMAT (hierarchical) =  $T_{\text{cache}} + (1 - h) \times T_{\text{memory}} = 2 + 0.04 \times 80 = 2 + 3.2 = \mathbf{5.2 \text{ ns}}$ .

4096 blocks; 1024 sets; address tag 13 / set 10 / offset 7; AMAT = 5.2 ns. Higher associativity cuts conflict misses but needs more tag comparators, so beyond a point it adds cost and delay for little gain.

### WORKED EXAMPLE

PQ 23/24 Q1D

An 8-way set-associative cache has 32-bit words, 4 words per line and 4096 sets. Find the size of each line, each set, and the whole cache.

- 1 Line size = 4 words  $\times$  32 bits = 4  $\times$  4 bytes = **16 bytes**.
- 2 Set size = 8 lines  $\times$  16 bytes = **128 bytes**.
- 3 Cache size = 4096 sets  $\times$  128 bytes = 524 288 bytes = **512 KB**.

Line 16 bytes, set 128 bytes, cache 512 KB.

### WORKED EXAMPLE

PQ 24/25 Q1B · PQ 23/24 Q3A

Cache hit ratio 80%, cache access 5 ns, main memory 100 ns. Find the average memory access time for (i) simultaneous and (ii) hierarchical organisation.

- 1 Simultaneous: AMAT =  $h \times T_c + (1 - h) \times T_m = 0.8 \times 5 + 0.2 \times 100 = 4 + 20 = \mathbf{24 \text{ ns}}$ .
- 2 Hierarchical: AMAT =  $T_c + (1 - h) \times T_m = 5 + 0.2 \times 100 = 5 + 20 = \mathbf{25 \text{ ns}}$ .

Simultaneous 24 ns, hierarchical 25 ns. Hierarchical pays the cache time on every access (even misses), so it is slightly slower here.

**Now redo it:** the difference is only whether the cache time is always paid (hierarchical) or only on hits (simultaneous). Learn both formulas.

## Elements of cache design

Beyond mapping, a few design decisions round out the topic and appear as short questions.

## TEACH

## THE CACHE READ OPERATION

On every read the CPU puts out an address and the cache checks it. **On a hit**, the cache returns the word straight to the CPU and **no traffic goes onto the system bus** at all, which is why a hit is fast. **On a miss**, the address is driven onto the system bus, main memory returns the whole block, and it is delivered to the cache and to the CPU together (the fetch and the fill happen in parallel), so the waited-for word arrives as soon as it can.

## TEACH

## PHYSICAL AND LOGICAL (VIRTUAL) ADDRESSES

Programs use **logical (virtual) addresses**; the hardware stores data at **physical addresses**. A **memory management unit (MMU)** translates one to the other through a page table, which is what lets each program behave as if it owns all of memory (**virtual memory**). A cache can be indexed by either address:

- ▶ A **physical cache** waits for the MMU to translate the address, then looks up the physical address. Simple and always correct.
- ▶ A **logical (virtual) cache** looks up the virtual address directly, in parallel with translation, so it is faster. The cost: different programs reuse the same virtual addresses, so the cache must be **flushed on every context switch**, or tagged with an address-space identifier, to avoid handing one program another's data.

## TEACH

## CHOOSING THE CACHE SIZE

Cache size is a balance. Too small and the hit rate falls and the CPU stalls; too large and it costs more, uses scarce chip or board area, and the extra addressing logic makes each lookup slightly slower. So cache is sized large enough for a good hit rate yet small enough to stay fast and affordable, which is exactly why machines use several small-to-large levels (L1, L2, L3) rather than one big cache.

## ⚠ TRAP

Two common errors. (1) Watch the **units** of the block: "16 words" with a 4-byte word is 64 bytes, so the offset counts words or bytes depending on how the address is given. (2) For set-associative, divide the lines by the associativity to get the **sets** before taking the log for the set bits; the extra bits move into the **tag**, not away.

## RECALL AND CHECK

**A direct-mapped cache has 1024 lines and 32-byte blocks, on a 32-bit machine. How many tag, index and offset bits?**

-----  
ANSWER

Offset =  $\log_2 32 = 5$ , index =  $\log_2 1024 = 10$ , tag =  $32 - 10 - 5 = 17$  bits.

### FILL IN THE GAP

1. The number of cache lines equals cache size divided by \_\_\_\_\_.
2. In direct mapping a block maps to line number equal to block number \_\_\_\_\_ the number of lines.
3. Offset bits = log base 2 of the \_\_\_\_\_ size.
4. Going from direct to k-way set associative moves bits from the index into the \_\_\_\_\_.

---

### ANSWERS

1. block size. 2. modulo (mod). 3. block. 4. tag.

## MODULE FOUR

# Basic and Advanced Processor Architectures

*Two design philosophies, the reduced instruction set (RISC) and the complex instruction set (CISC), and the ways many processors are joined to work in parallel. Mostly a describing-and-comparing module, which means fast, reliable marks if you know the contrasts cleanly.*

RISC

vs

CISC

## RISC versus CISC

*Two philosophies of instruction-set design: keep instructions few and simple, or make them many and powerful. Naming the differences and example processors is worth easy marks every year.*

*By the end of this unit you can give at least four differences between RISC and CISC and name example processors of each.*

### TEACH

### THE TWO PHILOSOPHIES

**CISC (Complex Instruction Set Computer)** provides many powerful instructions, some doing a lot of work in one instruction (including direct memory-to-memory operations). This makes programs short but the hardware complex, and instructions take varying numbers of cycles.

**RISC (Reduced Instruction Set Computer)** provides few, simple, fixed-length instructions, each doing a little in (ideally) one cycle, with memory touched only by dedicated load and store instructions (a **load-store** architecture). Programs are longer but the hardware is simple and fast, and it pipelines well.

| RISC                                     | CISC                                     |
|------------------------------------------|------------------------------------------|
| Few, simple instructions                 | Many, complex instructions               |
| Fixed-length instructions                | Variable-length instructions             |
| Mostly one cycle per instruction         | Instructions take several cycles         |
| Load-store: only load/store touch memory | Many instructions access memory directly |
| Many general-purpose registers           | Fewer registers                          |
| Simple hardware, easy to pipeline        | Complex hardware, harder to pipeline     |
| Complexity handled in the compiler       | Complexity handled in the hardware       |

### WORKED EXAMPLE

PQ 24/25 Q4A · PQ 20/21 Q4B · PQ 23/24 Q5C

State four differences between RISC and CISC, and give two example processors of each.

Four differences (any four): RISC has few simple fixed-length one-cycle instructions and only load/store touch memory, with many registers and simple pipelined hardware; CISC has many complex variable-length multi-cycle instructions, many of which access memory directly, with fewer registers and complex hardware. Examples: RISC, ARM and MIPS (also SPARC, PowerPC); CISC, Intel x86 (8086, Pentium) and the Motorola 68000.

**Now redo it:** pick your four contrasts (instruction count, length, cycles, memory access) and memorise two examples each: ARM/MIPS for RISC, x86/68000 for CISC.

**RISC:** ARM, MIPS, SPARC, PowerPC. **CISC:** Intel x86 (8086, Pentium), Motorola 68000.

**Memory hook:** the phone in your pocket is **ARM (RISC)**; the older desktop is **x86 (CISC)**.

## RECALL AND CHECK

**Why does RISC pipeline more easily than CISC?**

ANSWER

RISC instructions are fixed-length and mostly take one cycle, so the stages line up uniformly and hazards are easier to manage. CISC instructions vary in length and cycle count, which makes overlapping them much harder.

## MODULE FOUR · UNIT 2

## Parallel Architectures and Interconnection Networks

*When one processor is not enough, many are joined to work together. This unit classifies parallel machines and the networks that connect them, and completes the data-hazard picture from pipelining.*

*By the end of this unit you can state Flynn's taxonomy, describe common interconnection networks, and name the three types of data hazard.*

## TEACH

## FLYNN'S TAXONOMY

Parallel machines are classified by how many instruction streams and data streams they run at once:

- ▶ **SISD:** Single Instruction, Single Data. The classic single processor.
- ▶ **SIMD:** Single Instruction, Multiple Data. One instruction acts on many data items at once (vector and graphics processors).
- ▶ **MISD:** Multiple Instruction, Single Data. Rare; several operations on one data stream (some fault-tolerant designs).
- ▶ **MIMD:** Multiple Instruction, Multiple Data. Many processors running different instructions on different data (multicore CPUs, clusters).

## TEACH

## INTERCONNECTION NETWORKS

The processors and memories must be wired together. Common schemes: **shared bus** (simple, but one at a time), **crossbar** (every processor to every memory, fast but costly), **mesh** (each node to its neighbours, scales well), and **hypercube** and **ring** topologies. The trade-off is always cost and wiring against how many transfers can happen at once.

Completing the pipelining picture (Module Two), data hazards come in three types, named by the order of the clashing accesses:

- ▶ **RAW (read after write):** an instruction reads a value before the earlier instruction has written it. The true dependency, and the common one.
- ▶ **WAR (write after read):** an instruction writes a value before an earlier one has read it.
- ▶ **WAW (write after write):** two instructions write the same location in the wrong order.

## WORKED EXAMPLE

PQ 20/21 Q1C

Name the three types of data hazard.

Read after write (RAW), write after read (WAR), and write after write (WAW). RAW is a true dependency; WAR and WAW are name dependencies that arise when instructions are reordered.

## RECALL AND CHECK

**A graphics processor applies the same operation to thousands of pixels at once. Which Flynn category is it?**

ANSWER

SIMD: a single instruction acting on multiple data items simultaneously.

## S U P P L E M E N T

# Digital Logic and Boolean Algebra

*The gates and algebra beneath every processor. This material is not a unit of the current course outline, but older past papers set questions on it, so it is taught here to a full pass in case your paper does.*

$$F = AB + AC + BC \quad = \text{majority}(A,B,C)$$

---



# Boolean Algebra and K-maps

*Simplifying logic expressions, by algebra and by Karnaugh map, and turning them into gate circuits.*

## ⚠ NOT IN THE CURRENT SYLLABUS

The 2024/25 and 2025/26 papers do **not** ask Boolean algebra or K-maps. Older papers (2020/21, 2023/24) do. Learn this only after the graded topics are secure; it is here so no past question is left unanswerable.

## The operations and the laws

### TEACH

### BOOLEAN BASICS

Three operations: **AND** (written as a product,  $AB$ , true only if both), **OR** (written as a sum,  $A+B$ , true if either), and **NOT** (written  $A'$  or  $A$  with a bar, the opposite). The useful laws for simplifying:

- ▶ **Identity:**  $A + 0 = A$ ,  $A \cdot 1 = A$ . **Null:**  $A + 1 = 1$ ,  $A \cdot 0 = 0$ .
- ▶ **Complement:**  $A + A' = 1$ ,  $A \cdot A' = 0$ .
- ▶ **Absorption:**  $A + AB = A$ , and  $A + A'B = A + B$ .
- ▶ **De Morgan:**  $(AB)' = A' + B'$ , and  $(A + B)' = A'B'$ .

### WORKED EXAMPLE

### METHOD

Simplify  $F = A'BC + AB'C + ABC' + ABC$  by algebra.

- 1 Group the last two by  $AB$ :  $ABC' + ABC = AB(C' + C) = AB$ .
- 2 Take  $ABC$  again (it can be reused) with  $AB'C$  by  $AC$ :  $AB'C + ABC = AC(B' + B) = AC$ .
- 3 Take  $ABC$  again with  $A'BC$  by  $BC$ :  $A'BC + ABC = BC(A' + A) = BC$ .
- 4 So  $F = AB + AC + BC$ .

$F = AB + AC + BC$ . This is the **majority function**: it is 1 when at least two of  $A$ ,  $B$ ,  $C$  are 1.

**Now redo it:** the trick is that a minterm may be reused in more than one group. That is exactly what a K-map makes visual, next.

## The Karnaugh map

A K-map arranges the truth table so that cells differing in one variable are neighbours. Circle groups of 1s in powers of two (1, 2, 4, 8), as large as possible; each group drops the variable that changes across it. The circled groups, ORed together, are the simplified expression.

## WORKED EXAMPLE

## METHOD

Simplify the same  $F$  on a 3-variable K-map (rows  $A$ , columns  $BC$  in Gray-code order).  $F = 1$  at minterms 3, 5, 6, 7.

| BC  | 00 | 01 | 11 | 10 |
|-----|----|----|----|----|
| A=0 | 0  | 0  | 1  | 0  |
| A=1 | 0  | 1  | 1  | 1  |

Three pairs:  $BC$  (amber, both rows),  $AC$  (green),  $AB$  (teal).  $F = AB + AC + BC$ .

The three circled pairs give  $BC$ ,  $AC$  and  $AB$ , so  $F = AB + AC + BC$ , the same answer as the algebra, reached by eye.

## TEACH

## FROM EXPRESSION TO CIRCUIT

To draw the logic circuit for  $F = AB + AC + BC$ : one **AND** gate per product term ( $AB$ ,  $AC$ ,  $BC$ ), then one **OR** gate combining the three AND outputs into  $F$ . A sum-of-products expression always becomes a layer of AND gates feeding one OR gate. **Don't-care** outputs (marked d or X) may be treated as 0 or 1, whichever makes a larger group, so use them to simplify further.

## RECALL AND CHECK

**On a K-map, what does a group of two adjacent 1s tell you, and why circle the largest groups?**

ANSWER

A group of two drops the one variable that changes across it, leaving a product of the rest. Larger groups drop more variables, giving fewer and simpler terms, so always circle the biggest power-of-two groups you can.

## MOCK EXAMINATION ONE

# Sit This Paper Under Exam Conditions

*Two hours, no notes, one sheet of paper. This paper is weighted towards the 2024/25 and 2025/26 style: performance, pipelining, cache and floating point. Answer Question One and any other two. Full worked answers follow, so attempt everything before you turn to them.*



2 hours  
70 marks

Q1 (30) compulsory  
+ any two of 20

---

# Question Paper

Time: 2 hours. Answer **Question One (compulsory, 30 marks)** and **any other two questions (20 marks each)**. Total 70 marks.

## QUESTION ONE

**COMPULSORY · 30 MARKS**

- (a)** (i) Differentiate computer architecture from computer organization. (2) (ii) Compare the Von Neumann and Harvard architectures, with a diagram for each. (3)
- (b)** A processor runs 2 million instructions with this mix and cycle cost: ALU 45% at 1 cycle, Load 25% at 4 cycles, Store 15% at 2 cycles, Branch 15% at 3 cycles. (i) Find the average CPI. (ii) Find the total clock cycles. (iii) If Load is optimised to 2 cycles, find the new CPI. (iv) Find the percentage performance improvement. (6)
- (c)** Using a **12-bit** register, represent **150** and **-95** in unsigned, sign-magnitude, one's complement and two's complement. (6)
- (d)** (i) Represent **39.5** in IEEE-754 single precision, giving the hex. (ii) Find the decimal value of **42480000H** under IEEE-754. (9)
- (e)** A 32-bit machine has a direct-mapped cache of 1024 lines with 32-byte blocks. Give the number of tag, index and offset bits. (4)

## QUESTION TWO

**20 MARKS**

- (a)** A 5-stage pipeline has stage times 20, 15, 25, 20 and 10 ns with a 5 ns latch delay. For 1000 tasks find (i) the cycle time, (ii) the non-pipeline time for one task, (iii) the pipeline time for 1000 tasks, (iv) the sequential time for 1000 tasks, (v) the speedup. (10)
- (b)** Explain the three kinds of pipeline hazard, and say how forwarding and a single stall handle a load-use data hazard. (6)
- (c)** Why is the speedup of a real pipeline always less than the number of stages? (4)

## QUESTION THREE

**20 MARKS**

- (a)** A system has 256 MB main memory, a 64 KB cache with 32-byte blocks, using 4-way set associative mapping. Find (i) the number of cache blocks, (ii) the number of sets, (iii) the tag, set and offset field widths. (8)
- (b)** The cache hit ratio is 90%, cache access is 3 ns and main memory is 60 ns. Find the average memory access time for simultaneous and for hierarchical organisation. (6)
- (c)** Explain locality of reference, distinguishing temporal from spatial locality, and say how each shapes cache design. (6)

**QUESTION FOUR****20 MARKS**

- (a) Write  $Y = (A + B) \times (C - D)$  in the 3-address, 2-address, 1-address and 0-address instruction formats. (10)
- (b) Given  $PC = 4000$ ,  $R1 = 2500$ , an address field of 3200, and memory where  $M[1500] = 700$ ,  $M[2500] = 900$ ,  $M[3200] = 1500$  and  $M[7200] = 350$ , give the effective address and operand for direct, indirect, register-indirect (R1) and relative addressing. (6)
- (c) State four differences between RISC and CISC, with one example processor of each. (4)

**QUESTION FIVE****20 MARKS**

- (a) State Amdahl's law. A program has 90% of its work parallelised to run 8 times faster. Find the overall speedup. (6)
- (b) State four differences between DRAM and SRAM, and say which is used for cache and which for main memory, and why. (8)
- (c) Describe the four memory access methods, with one example device each. (6)

**MOCK EXAMINATION ONE**

## Answers

*Full worked solutions. Each cross-refers to the unit that teaches it, so a slip points you straight to the revision you need.*

**(A) ARCHITECTURE VS ORGANIZATION; VON NEUMANN VS HARVARD (SEE M1 U1)**

Architecture is the programmer-visible design (instruction set, registers, addressing modes): what the machine does. Organization is how that is built in hardware (control signals, buses, memory technology): how it does it. Von Neumann uses one memory and one bus for instructions and data (simple, but the bus is a bottleneck); Harvard uses separate memories and buses (simultaneous access, faster, costlier). Draw both diagrams.

**(B) CPI (SEE M2 U2)**

(i)  $CPI = 0.45(1) + 0.25(4) + 0.15(2) + 0.15(3) = 0.45 + 1.00 + 0.30 + 0.45 = \mathbf{2.20}$ . (ii) Total cycles =  $2\,000\,000 \times 2.20 = \mathbf{4\,400\,000}$ . (iii) With Load at 2 cycles:  $CPI = 0.45 + 0.25(2) + 0.30 + 0.45 = \mathbf{1.70}$ . (iv) Improvement =  $(2.20 - 1.70) / 2.20 = \mathbf{22.73\%}$ .

**(C) 12-BIT REPRESENTATIONS (SEE M1 U2)**

150 = 0000 1001 0110 (positive, so identical in all four schemes). For -95: +95 = 0000 0101 1111.

| VALUE | UNSIGNED          | SIGN-MAGNITUDE | ONE'S COMPLEMENT | TWO'S COMPLEMENT |
|-------|-------------------|----------------|------------------|------------------|
| 150   | 0000 1001 0110    | 0000 1001 0110 | 0000 1001 0110   | 0000 1001 0110   |
| -95   | not representable | 1000 0101 1111 | 1111 1010 0000   | 1111 1010 0001   |

**(D) IEEE-754 (SEE M1 U3)**

(i)  $39.5 = 100111.1 = 1.001111 \times 2^5$ . Exponent  $5 + 127 = 132 = 1000\,0100$ . Sign 0. Mantissa 001111 padded to 23 bits. Word = 0 10000100 001111000000000000000000 = **421E0000H**. (ii) 42480000H = 0100 0010 0100 1000...: sign 0, exponent 1000 0100 = 132 so  $e = 5$ , mantissa 1001..., value =  $1.1001 \times 2^5 = 110010 = \mathbf{50.0}$ .

**(E) CACHE FIELDS (SEE M3 U3)**

Offset =  $\log_2 32 = 5$  bits; index =  $\log_2 1024 = 10$  bits; tag =  $32 - 10 - 5 = \mathbf{17\ bits}$ .

**(A) PIPELINE TIMING (SEE M2 U3)**

(i) Cycle time = slowest stage + latch =  $25 + 5 = 30$  ns. (ii) Non-pipeline one task =  $20 + 15 + 25 + 20 + 10 = 90$  ns. (iii) Pipeline for 1000 tasks =  $(5 + 999) \times 30 = 1004 \times 30 = 30\,120$  ns. (iv) Sequential for 1000 =  $1000 \times 90 = 90\,000$  ns. (v) Speedup =  $90\,000 / 30\,120 = 2.99$ .

**(B) HAZARDS (SEE M2 U3, M4 U2)**

Structural (two instructions need the same hardware: duplicate it), data (a value is needed before it is written back: use forwarding, and one stall for a load-use case), control (a branch changes the next instruction: resolve early or flush). Forwarding feeds a result straight from where it is produced; a load value is not ready until the memory stage, so the very next instruction still needs one stall.

**(C) WHY SPEEDUP < STAGES**

The pipeline must first fill (the first task still takes  $k$  cycles), and hazards insert stalls, so the ideal ceiling of  $k$  is never quite reached, especially for few tasks.

**(A) CACHE CONFIGURATION (SEE M3 U3)**

(i) Blocks = cache / block =  $64\text{ KB} / 32\text{ B} = 2^{16} / 2^5 = 2048$  blocks. (ii) Sets =  $2048 / 4 = 512$  sets. (iii) Address =  $256\text{ MB} = 2^{28}$ , so 28 bits. Offset =  $\log_2 32 = 5$ , set =  $\log_2 512 = 9$ , tag =  $28 - 9 - 5 = 14$  bits. Format: tag 14, set 9, offset 5.

**(B) AMAT (SEE M3 U3)**

Simultaneous =  $0.9(3) + 0.1(60) = 2.7 + 6 = 8.7$  ns. Hierarchical =  $3 + 0.1(60) = 3 + 6 = 9.0$  ns.

**(C) LOCALITY (SEE M3 U1)**

Locality of reference is the clustering of accesses in a small region over any short time. Temporal: a location used now is likely used again soon (so cache keeps recently used data). Spatial: nearby locations are likely used soon (so cache fetches whole blocks of neighbours).

**(A)  $Y = (A + B) \times (C - D)$  IN FOUR FORMATS (SEE M1 U4)**

| 3-ADDRESS   | 2-ADDRESS | 1-ADDRESS | 0-ADDRESS |
|-------------|-----------|-----------|-----------|
| ADD T1,A,B  | MOV T1,A  | LOAD A    | PUSH A    |
| SUB T2,C,D  | ADD T1,B  | ADD B     | PUSH B    |
| MUL Y,T1,T2 | MOV T2,C  | STORE T1  | ADD       |
|             | SUB T2,D  | LOAD C    | PUSH C    |
|             | MUL T1,T2 | SUB D     | PUSH D    |
|             | MOV Y,T1  | MUL T1    | SUB       |
|             |           | STORE Y   | MUL       |
|             |           |           | POP Y     |

**(B) ADDRESSING MODES (SEE M1 U4)**

| MODE                   | EFFECTIVE ADDRESS | OPERAND        |
|------------------------|-------------------|----------------|
| Direct                 | 3200              | M[3200] = 1500 |
| Indirect               | M[3200] = 1500    | M[1500] = 700  |
| Register-indirect (R1) | R1 = 2500         | M[2500] = 900  |
| Relative               | PC + 3200 = 7200  | M[7200] = 350  |

**(C) RISC VS CISC (SEE M4 U1)**

RISC: few simple fixed-length one-cycle instructions, load-store, many registers (example ARM or MIPS). CISC: many complex variable-length multi-cycle instructions, direct memory access, fewer registers (example Intel x86 or Motorola 68000).

**(A) AMDAHL'S LAW (SEE M2 U2)**

Speedup =  $1 / ((1 - f) + f / s)$ . With  $f = 0.90$  and  $s = 8$ : Speedup =  $1 / (0.10 + 0.90 / 8)$  =  $1 / (0.10 + 0.1125)$  =  $1 / 0.2125$  = **4.71**.

**(B) DRAM VS SRAM (SEE M3 U2)**

DRAM stores a bit as capacitor charge, needs refresh, is slower, denser and cheaper, and is used for **main memory**. SRAM stores a bit in a transistor latch, needs no refresh, is faster, less dense and dearer, and is used for **cache**. Cache needs speed; main memory needs cheap capacity.

**(C) MEMORY ACCESS METHODS (SEE M3 U1)**

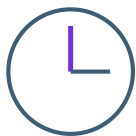
Sequential (read in order, example magnetic tape), direct (jump near then search, example disk), random (any location in equal time, example RAM), associative (located by content, example a cache tag store).



## MOCK EXAMINATION TWO

# Sit This One After the First

*A second full paper in the same shape, leaning to the 2023/24 and 2024/25 style: number representation, floating point, cache mapping and instruction formats. Two hours. Answer Question One and any other two. Solutions follow.*



2 hours  
70 marks

Q1 (30) compulsory  
+ any two of 20

---

# Question Paper

Time: 2 hours. Answer **Question One (compulsory, 30 marks)** and **any other two questions (20 marks each)**. Total 70 marks.

## QUESTION ONE

**COMPULSORY · 30 MARKS**

- (a) (i) State the Von Neumann concept and two of its significant aspects. (3) (ii) What is the Von Neumann bottleneck, and give two ways to reduce it? (2)
- (b) A 16-bit two's complement processor stores 300, -200 and -5000. (i) Give the representable range and confirm all three fit. (ii) Represent each in binary and hexadecimal. (iii) Compute  $300 + (-200)$  in two's complement. (9)
- (c) (i) Represent **85.75** in IEEE-754 single precision (hex). (ii) Find the decimal value of **C2960000H** under IEEE-754. (8)
- (d) Write  $Z = (A + B) \times (C + D) + E$  in the 3-address and 0-address formats, and count the bytes of each assuming 1 byte per opcode and 2 bytes per address. (8)

## QUESTION TWO

**20 MARKS**

- (a) A 32-bit machine has a direct-mapped cache in which each block holds 8 words and there are 512 blocks. (i) Give the offset, index and tag field widths. (ii) In which cache block do the addresses ABCD1234H and 5678FEDCH fall? (10)
- (b) Explain the three cache mapping functions (direct, fully associative, set associative), with one advantage and one disadvantage of each. (10)

## QUESTION THREE

**20 MARKS**

- (a) Using a **10-bit** register, complete the unsigned, sign-magnitude, one's complement and two's complement forms of **200** and **-75**. (8)
- (b) Using end-around carry in one's complement, compute  $200 - 75$  in a 10-bit register. (6)
- (c) Find the memory needed for a black-and-white image of 8 inches by 10 inches at 200 dpi. Give the answer in megabytes. (6)

## QUESTION FOUR

**20 MARKS**

- (a) Find the decimal value of **43A00000H** under IEEE-754 single precision. (6)
- (b) Name the three categories of instructions, with two examples of each. (6)
- (c) Describe the immediate, direct, indirect and register-indirect addressing modes, saying where the operand is in each. (8)

## QUESTION FIVE

20 MARKS

- (a) A 4-stage pipeline has stage times 40, 30, 50 and 35 ns with a 5 ns latch. For 200 tasks, find the cycle time, the pipeline time, the sequential time and the speedup. (8)
- (b) Draw and describe the memory hierarchy, from registers to secondary storage, saying how speed, size and cost change down it. (6)
- (c) A cache has an 85% hit ratio, 4 ns cache access and 70 ns main memory access. Find the average memory access time for a hierarchical organisation. (6)

## MOCK EXAMINATION TWO

# Answers

Full worked solutions, each cross-referenced to the unit that teaches it.

## ANSWER

## QUESTION ONE

### (A) VON NEUMANN (SEE M1 U1)

(i) The Von Neumann concept: a stored-program computer keeps instructions and data in the same memory and executes instructions sequentially. Two aspects: a single shared memory for code and data; sequential fetch-execute of a stored program. (ii) The bottleneck: the single shared bus lets only instructions or data move at a time, so the CPU waits on memory. Reduce it with cache and with separate instruction and data caches (or wider buses).

### (B) 16-BIT TWO'S COMPLEMENT (SEE M1 U2)

(i) Range =  $-2^{15}$  to  $+(2^{15}-1) = -32768$  to  $32767$ ; 300, -200 and -5000 all fit.

| VALUE | BINARY (16-BIT TWO'S COMPLEMENT) | HEX  |
|-------|----------------------------------|------|
| 300   | 0000 0001 0010 1100              | 012C |
| -200  | 1111 1111 0011 1000              | FF38 |
| -5000 | 1110 1100 0111 1000              | EC78 |

(iii)  $300 + (-200)$ : add the patterns  $012C + FF38 = 1\ 0064$ , discard the carry out of bit 16, leaving  $0064H = \mathbf{100}$ . Check:  $300 - 200 = 100$ .

### (C) IEEE-754 (SEE M1 U3)

(i)  $85.75 = 1010101.11 = 1.01010111 \times 2^6$ . Exponent  $6 + 127 = 133 = 1000\ 0101$ . Mantissa  $01010111$  to 23 bits. Word =  $0\ 10000101\ 010101110000000000000000 = \mathbf{42AB8000H}$ . (ii)  $C2960000H$ : sign 1, exponent  $1000\ 0101 = 133$  so  $e = 6$ , mantissa  $00101110$ , value =  $-1.0010110 \times 2^6 = -1001011 = \mathbf{-75.0}$ .

### (D) INSTRUCTION FORMATS AND BYTES (SEE M1 U4)

3-address (opcode 1, three addresses 2 each = 7 bytes each):

|              |              |                |              |
|--------------|--------------|----------------|--------------|
| ADD T1, A, B | ADD T2, C, D | MUL T1, T1, T2 | ADD Z, T1, E |
|--------------|--------------|----------------|--------------|

4 instructions  $\times 7 = \mathbf{28\ bytes}$ . 0-address (postfix  $A\ B + C\ D + \times E +$ ): PUSH A, PUSH B, ADD, PUSH C, PUSH D, ADD, MUL, PUSH E, ADD, POP Z = 10 instructions = 5 pushes  $\times 3 + 1\ pop \times 3 + 4\ ops \times 1 = \mathbf{22\ bytes}$ .

## ANSWER

## QUESTION TWO

**(A) CACHE FIELDS AND BLOCK LOOKUP (SEE M3 U3)**

(i) Offset =  $\log_2 8 = 3$ , index =  $\log_2 512 = 9$ , tag =  $32 - 9 - 3 = 20$ . (ii) Block = (address with the low 3 offset bits removed) mod 512, that is (address  $\div 8$ ) mod 512. ABCD1234H gives block **70**; 5678FEDCH gives block **475**.

**(B) THE THREE MAPPING FUNCTIONS (SEE M3 U3)**

Direct: one line per block (fast and simple, but conflict misses). Fully associative: a block may go anywhere (no conflict misses, but every tag must be searched, costly). Set associative: a block maps to one set of  $k$  lines (the compromise, fewer conflicts than direct at less cost than fully associative).

## ANSWER

## QUESTION THREE

**(A) 10-BIT REPRESENTATIONS (SEE M1 U2)**

200 = 0011001000 (positive, same in all four). For  $-75$ :  $+75 = 0001001011$ .

| VALUE | UNSIGNED          | SIGN-MAGNITUDE | ONE'S COMPLEMENT | TWO'S COMPLEMENT |
|-------|-------------------|----------------|------------------|------------------|
| 200   | 0011001000        | 0011001000     | 0011001000       | 0011001000       |
| $-75$ | not representable | 1001001011     | 1110110100       | 1110110101       |

**(B) END-AROUND CARRY (SEE M1 U2)**

$200 - 75 = 200 + (-75)$ .  $200 = 0011001000$ ;  $-75$  in one's complement = 1110110100. Add:  $0011001000 + 1110110100 = 1\ 0001111100$ , a carry out of bit 10. End-around: drop it and add 1 to the low bit:  $0001111100 + 1 = 0001111101 = 125$ . Check:  $200 - 75 = 125$ .

**(C) IMAGE MEMORY (SEE M1 U2)**

Pixels =  $(8 \times 200) \times (10 \times 200) = 1600 \times 2000 = 3\ 200\ 000$ . At 1 bit per pixel that is 3 200 000 bits = 400 000 bytes =  $400\ 000 / 1\ 048\ 576 = 0.381\ \text{MB}$  (about 0.38 MB).

## ANSWER

## QUESTION FOUR

**(A) IEEE-754 REVERSE (SEE M1 U3)**

43A00000H = 0100 0011 1010 0000...: sign 0, exponent 1000 0111 = 135 so  $e = 8$ , mantissa 0100000. Value =  $1.0100000 \times 2^8 = 101000000 = 320.0$ .

**(B) INSTRUCTION CATEGORIES (SEE M1 U4)**

Data transfer (LOAD, STORE, MOVE, PUSH, POP), arithmetic and logic (ADD, SUB, MUL, AND, OR), control or test-and-branch (JUMP, BEQ, CALL, RETURN).

**(C) ADDRESSING MODES (SEE M1 U4)**

Immediate: the operand is the address field itself (a constant). Direct: the operand is  $M[\text{address field}]$ . Indirect: the field points to a location that holds the address, so the operand is  $M[M[\text{address field}]]$ . Register-indirect: a register holds the address, so the operand is  $M[\text{register}]$ .

**(A) PIPELINE TIMING (SEE M2 U3)**

Cycle time = slowest stage + latch = 50 + 5 = **55 ns**. Pipeline time for 200 tasks =  $(4 + 199) \times 55 = 203 \times 55 = \mathbf{11\ 165\ ns}$ . Sequential time =  $200 \times (40 + 30 + 50 + 35) = 200 \times 155 = \mathbf{31\ 000\ ns}$ . Speedup =  $31\ 000 / 11\ 165 = \mathbf{2.78}$ .

**(B) MEMORY HIERARCHY (SEE M3 U1)**

Registers, then cache (L1, L2, L3), then main memory (RAM), then secondary storage (disk, SSD), then tertiary (tape). Going down: bigger, slower and cheaper per byte. Going up: smaller, faster and dearer.

**(C) AMAT (SEE M3 U3)**

Hierarchical:  $AMAT = T_{\text{cache}} + (1 - h) \times T_{\text{memory}} = 4 + 0.15 \times 70 = 4 + 10.5 = \mathbf{14.5\ ns}$ .